University of Cabuyao
(PAMANTASAN NG CABUYAO)

COLLEGE OF ENGINEERING

Optimizing Local Small Language Models Using 4-Bit Quantization, QLoRA, and RAG for University Helpdesks

A Computer Engineering Research Presented to
the Faculty of the College of Engineering
University of Cabuyao

In Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Engineering

By

Anciso, Wendell R.
Aranas, Charlyn Mae M.
Hermosilla, Ivan M.
Oliver, Aerial Iverson P.
Paculan, Michelle Caylin M.

May 2026

CHAPTER I

THE PROBLEM AND ITS SETTING

Introduction

University helpdesk systems offer students crucial information regarding enrollment processes, academic policies, student services, office directories and other matters concerning universities. Nevertheless, access to correct and up-to-date information is difficult when institutional resources are scattered among different documents, handbooks, and administrative channels. Students may be required to inquire using long documents or contact a number of offices, which may prolong the process of receiving a response to his/her inquiries or lead to numerous requests. As a result, a student may encounter difficulties in locating the necessary information when there is no support available from university representatives. A quantitative study conducted in the context of Higher Education in the Philippines among 198 Filipino graduate students points to students' problems with academic resources pertaining to library assistance and supervision [1].

This challenge is directly observed by researchers at the University of Cabuyao, a university located in Cabuyao City, Laguna, Philippines, concerned with providing quality education for students. Within the university environment, timely access to e-administrative support and academic services plays an important role in everyday contacts between students and the educational establishment. However, the use of online cloud-based systems may lead to various issues associated with connectivity problems, server failures, and expensive bandwidth requirements,

which demonstrates an obvious necessity of an offline system that would work on the premises of the institution [49].

This issue has been highlighted by a recent survey carried out among the university students. An overwhelming majority (more than 90%) of respondents rated their problem of finding university information with very high severity (4 or 5 out of 5). Students from all the years stated that they faced the problem of scattered information, out-of-date content, and the long lines in the offices all the time. Instead of having a proper channel, the majority of students have to rely on other people's experience and verify information with their friends. The results indicate that there is an urgent need for students to have a single source of high quality and convenient knowledge base.

One approach to addressing this need is the use of language-model-based chatbots that can respond to students automatically. Unfortunately, language models may generate inaccurate or unsupported information because they only rely on pre-trained knowledge. Small Language Models can be chosen as they can offer something more resource-efficient than larger Language Models allowing the use of SLMs when computing resources are scarce. A recent study regarding chatbots for academic purposes made possible the application of a compact LLaMA model alongside Retrieval-Augmented Generation [4]. Retrieval-Augmented Generation works to eliminate this issue as it allows the language model to acquire the necessary information from another knowledge base before crafting an answer. A recent survey in this field pointed out the significance of some aspects while dealing with RAG in education, especially accuracy, relevance, faithfulness, and quality of generated text [2]. Besides, a recent study about RAG in a university context reported that retrieval integration can help decrease the level of hallucination from

31.7 to 6.6%, as well as the use of quantized inference for limited computational items [3]. The results demonstrate that RAG may be used in order to achieve accurate responses from university helpdesks on the one hand while proving the necessity of being efficient when seeking the required computational resources.

The researchers propose the study titled “Optimizing Local Small Language Models Using 4-Bit Quantization, QLoRA, and RAG for University Helpdesks.” This study aims to develop and evaluate a controlled offline-capable university helpdesk prototype that generates evidence-grounded and citation-supported responses to English university-related queries, utilizing chosen student-facing institutional documents. The prototype is realized through combining Small Language Model software hosted on a local server with a 4-bit quantization technology, usage of QLoRA for behavioral adaptation, and RAG technology with hybrid information retrieval to obtain data concerning the university. Moreover, the research was designed to adopt a simple query-routing technique able to provide relevant responses with regard to the availability of the retrieved information.

Research Objectives

The main objective of this study is to develop and evaluate an offline university helpdesk chatbot that answers student inquiries from the university's official manuals with cited responses, using a 4-bit quantized Llama 3.2 3B model on a computer with 8 GB of VRAM.

1. Deploy the Llama 3.2 3B model in a 4-bit quantized format for offline use, and evaluate its response time, speed, and memory.

2. Develop a knowledge base from the university's official student manuals, recording the document title, issuing office, year, and page of each passage for citation.

3. Apply QLoRA-based behavioral fine-tuning and compare the adapted model with the unadapted 4-bit model in terms of answer correctness, instruction following, and handling of unsupported queries.

4. Implement a Retrieval-Augmented Generation pipeline that retrieves relevant passages from the knowledge base and generates answers with citations, and evaluates its retrieval accuracy, groundedness, and citation accuracy.

5. Implement a query-routing mechanism that decides between a cited answer, an office referral, an out-of-scope message, and evaluate its routing accuracy.

6. Evaluate the prototype's functional suitability, performance efficiency, and reliability using selected characteristics from ISO/IEC 25010:2023.

Theoretical Framework

This study is guided by model quantization, parameter-efficient fine-tuning, knowledge-based systems, Retrieval-Augmented Generation (RAG), query routing, and software quality evaluation. These theoretical bases explain and support the major design and evaluation choices of the proposed offline university helpdesk chatbot.

Model Compression and Quantization

The process of quantizing a model minimizes the amount of memory space a language model utilizes by encoding its weights on lesser bits while still being able to retain almost the same level of efficacy [5]. Lin et al. [7], in their work on activation-aware weight quantization, show that low-bit quantization can still support effective on-device deployment of a language model by protecting the small fraction of weights most important to model performance. However, Meta [6] also points out that Llama 3.2 3B is a very efficient model designed for the specialty of instruction tuning, hence able to perform well across multiple languages. Together, these findings support selecting the 4-bit version of Llama 3.2 3B for this project. The model will be deployed locally and evaluated in terms of response time, generation speed, and memory use under the available 8 GB VRAM limit.

Parameter-Efficient Fine-Tuning through QLoRA

QLoRA enables a quantized language model to be fine-tuned through small trainable low-rank adapters instead of updating all of its parameters [8]. This approach supports the study's use of fine-tuning under limited computing resources. In this study, QLoRA will be used to reinforce helpdesk-specific behaviors, including

direct answering, required response formatting, citation provision, and proper handling of questions that lack sufficient supporting evidence. Its effect will be measured by comparing the adapted model with the unadapted 4-bit model on held-out questions, looking at answer correctness, instruction following, and handling of unsupported queries.

Knowledge-Based Systems and Document Provenance

A knowledge-based system depends on an organized collection of domain-specific information that the system can use when responding to a query. In this study, the knowledge base will consist only of approved university student manuals. Each indexed passage will retain its document title, issuing office, year, and page number. This supports source traceability by allowing the system and evaluators to connect generated claims and citations to the institutional document from which the information was retrieved.

Retrieval-Augmented Generation Architecture

In 2020, Lewis et al. [9] introduced Retrieval-Augmented Generation (RAG), a framework that connects a retrieval tool with a generator so that a language model could generate text based on data retrieved from an external source of knowledge. This justifies the decision made in the current study to draw the resulting answer from the retrieved sources rather than purely from the internal knowledge of the model. In the framework of this project, the answers will be generated with the help of the passages presented in the university-approved manuals for the students. Moreover, in order to examine the effectiveness of the pipeline, the study analyzes the results of RAG by focusing on its retrieval effectiveness, answer grounding level, and citation accuracy using an evaluation procedure specially developed for this

project and tested on a human dataset. This LLM-as-judge approach follows the same general principle used in RAG evaluation frameworks such as RAGAS [10], though the specific metrics and rubric used here were developed for this study rather than adopted directly from RAGAS.

Query Routing and Selective Response

A university helpdesk receives different types of questions, and not all of them should be answered through document retrieval. This contributes to the implementation of a routing mechanism that would select the proper response path based on the question and evidence. In this study, questions would be routed either to a cited answer or an office referral, or an out of scope message. The rate of routing will be measured against the expected routing in the evaluation set.

Geifman and El-Yaniv [12] demonstrate that through selective classification, systems may reject or abstain predictive decisions in case they lack confidence in their predictions, which might help mitigate the chances of generating unreliable results. Similarly, studies on unanswerable question answering such as Rajpurkar et al. [13], emphasize the importance of recognizing when available context is insufficient to provide a valid response. Adapting this concept, the prototype applies selective classification to its retrieval scores rather than to the model's own confidence. The router compares retrieval-score thresholds and a labeled-exemplar lookup, both decided before the language model is ever called. Instead of being forced to produce potentially incorrect information when the evidence gathered is inadequate, the system refers the user to either an office referral or an out-of-scope communication. Hence, the output will be reliable and accurate because the evidence is sufficient.

ISO/IEC 25010:2023 Software Product Quality Model

ISO/IEC 25010:2023 provides a recognized model for specifying and evaluating software product quality [7]. The adopted model in this study assists in overall understanding of the performance of the actual chatbot prototype. Functional suitability, performance efficiency, and reliability characteristics will be applied to evaluate whether this system meets its helpdesk functions, uses the computing resources efficiently, and maintains its regular performance under the conditions of the study.

Taken together, these theoretical bases support the connection between the study's design and objectives. Quantization supports offline deployment, QLoRA supports behavioral adaptation, document provenance supports traceable sources, RAG supports evidence-grounded generation, query routing supports appropriate response selection, and ISO/IEC 25010:2023 supports the overall evaluation of the prototype.

Conceptual Framework

This part of the study illustrates how the researchers intend to flow through the study by implementing the theoretical framework into a functional system design.

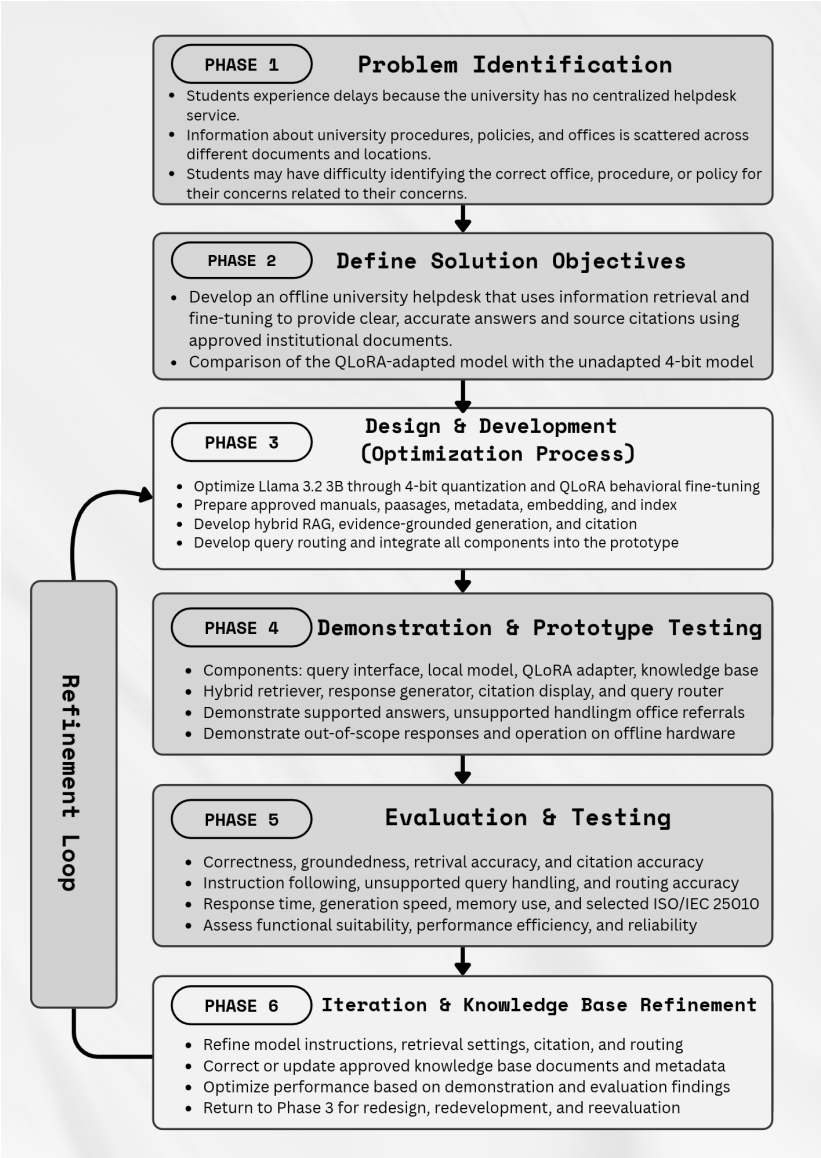


Figure 1. Paradigm of the Conceptual Framework

Figure 1 shows that this study relies on the Design Science Research Methodology (DSRM) for the systematic development, demonstration, evaluation, and refinement of an offline university helpdesk. The methodology is divided into six interconnected phases: problem identification; definition of solution requirements; designing and building; demonstration and testing; evaluation and testing; and iterations and improvement of the knowledge base.

The **Problem Identification** phase focuses on the delays students experience in obtaining timely, accurate, and accessible information because, as observed by the researchers, there is no centralized helpdesk at University of Cabuyao. Students often need to review extensive student manuals and other approved institutional documents or contact multiple offices for concerns regarding enrollment, academic policies, student services, office directories, and institutional procedures. Moreover, existing language models might produce incorrect or unsupported information since their responses are not grounded authorized institutional sources. All of these issues justify the need to create a resource-efficient as well as offline capable university helpdesk prototype.

The **Definition of Solution Objectives** phase establishes the intended capabilities of the proposed University of Cabuyao helpdesk. It aims to develop and provide a resource efficient and offline system that implements features of fine-tuning with information retrieval to provide accurate, relevant, and evidence-based answers from authorized institutional documents. The proposed solution is intended to respond appropriately to students' inquiries, include traceable source citations and clearly communicate when a question is unclear, outside its scope, or unsupported by the available information. It also defines the system's functional suitability, performance efficiency, and reliability objectives, and sets the comparison between

the QLoRA-adapted model and the unadapted 4-bit model that will show the effect of fine-tuning.

The third phase is **Design and Development.** This phase defines the procedure of optimization and integration of all elements of the system. It focuses on designing the overall structure of the proposed system and defining how its major components will work together. The process begins when a student submits a query, which is used to search the knowledge base for relevant passages. If the retrieved passages contain sufficient evidence, the system generates an evidence-grounded response with a traceable source citation. If the retrieved passages do not contain sufficient evidence, the system then checks whether the query falls within the subject matter of the three manuals: a concern that does, but lacks supporting evidence, receives an office referral; a query unrelated to the manuals receives an out-of-scope message. The final response will then be presented to the student through the system's query interface.

The fourth phase is **Demonstration and Prototype Testing,** during this phase the researchers will demonstrate how the prototype developed and test its capabilities, using relevant queries related to university related queries. The demonstration will include an explanation of how an inquiry from a student is received, determines the appropriate response path, searches the knowledge base, retrieves from university documents, generates an grounded response with help of sources which can be tracked down. The researchers will also show how the prototype provides office referrals when a concern requires institutional action, returns an appropriate message for an out-of-scope query, and operates under the designated offline hardware configuration.

The fifth phase, **Evaluation and Testing,** determines how well the prototype meets the established research objectives and evaluation criteria. Predefined questions, behaviorally evaluated data, as well as test cases will be used to evaluate how correct the answers are, instruction following, how well unsupported queries are processed, retrieval accuracy, response groundedness, citation accuracy, routing accuracy. The researchers will also compare the QLoRA-adapted model with the unadapted 4-bit model on the same questions to measure the effect of fine-tuning. In addition, researchers will assess the efficiency of the model working on the specified offline hardware condition; it also will be evaluated according to the selected functional suitability, performance efficiency, and reliability characteristic of ISO/IEC 25010:2023.

Lastly, the Iteration and Knowledge Base Refinement. In this phase, the results obtained during the demonstration and evaluation phases are used to improve the prototype. Adjustments to document preparation, passage segmentation, indexing, and retrieval setting might take place due to retrieval errors. Citation errors may lead to revision of the behavioral training, the system instructions, and evidence rules. Publishing and quotation mistakes may result in corrections in metadata and citation mapping, while routing inaccuracies might demand modification of routing rules and decision thresholds. Unavailable or outdated institutional data can be revised by improving the approved knowledge base as well. After the refinement, the components involved in this cycle will return to the design and development phase and undergo another cycle of demonstration and evaluation.

Significance of the Study

The goal of this study is to design and test an offline, locally hosted helpdesk system which can respond to questions concerning the university using authorized school literature. The purpose of this system is to refer to its sources, not make assumptions when it has insufficient information, safeguard classified or private data, and operate efficiently even on available limited university equipment. In addition, this study assesses the system using the predetermined good quality and efficient performance criteria. This is intended to reduce, over time, the need for manual staff handling of common questions, while still ensuring accuracy and trust in the answers given.

This study is significant to the following groups:

To the University Offices (e.g., registrar, admissions, student affairs) who are currently burdened by repetitive informational queries. An offline helpdesk that answers questions from an approved corpus of documents and points out their source can reduce the torrents of routine questions flowing to the staff. An abstention and redirection approach is intended to send questions that require human judgment or access to restricted information to the responsible office rather than answering them.

To Students, it provides convenience to the students in seeking immediate responses to questions about rules, policies, and procedures in school because they do not have to schedule a meeting with the administrator to do so. In addition, the prototype needs no internet connection to operate. It runs on one machine on site,

giving students immediate answers without much effort on their part. Thus, providing a human-like sense of assistance with the right, cited answers.

Regarding data privacy, the entire system uses hardware local to the test environment rather than third-party servers . Data belonging to students/institutions will be kept at the unit location; therefore, the institution will minimize the risk of revealing valuable information to other third parties.

To Future Researchers and Computer Engineering Students, students should consider this research while developing an offline, limited hardware-based, document-based assistant for their future research work. The configuration evaluated and the results obtained by the researchers provide a good guideline for other students working on a similar project in the future that requires limited hardware for its functioning.

To the Developers and builders. This study documents a repeatable procedure for updating the knowledge base as institutional documents change, providing a reference workflow that could inform future maintenance of a production system, though ongoing operation is outside this study's scope.

Assumption

The assumptions of this study are as follows:

1. The handbooks and manuals that came from the University of Cabuyao learning management system are official, correct, and show the university policy that was in effect when they were downloaded.
2. The Llama 3.2 3B model, after being reduced to 4-bit, retains the language ability needed to follow instructions and give clear answers, as described in its official model documentation and shown in prior evaluations of this model at 4-bit precision [14]. This is assumed rather than measured, because quantization is held constant across all test conditions.
3. The retrieval parts. BM25, embeddings, and Reciprocal Rank Fusion. Do their job when finding relevant parts of text, as they have been used before in retrieval research.
4. The 80 test queries used for testing are an example of the kinds of questions students might ask the system, even if they do not cover every way a question can be phrased or every topic.
5. The chosen ISO/IEC 25010:2023 characteristics are good enough to show the quality of the prototype even if not all parts of the standard are checked.
6. The test machine with 8 GB VRAM works as expected during testing without any hardware problems or slowdowns that could change the timing results.
7. The ten measures identified in the Research Objectives are assumed to provide an adequate basis for characterizing the performance of the prototype

within the limits of this study. No claim can be made about the precision, accuracy, or speed of the operation before the tests are carried out; all parameters are recorded and presented against the preset values.

Scope and Limitations

Scope and delimitations are boundaries established by the researchers, while limitations are constraints that may affect the study and are not always fully controllable.

Scope

1. The study covers the configuration of a 4-bit quantized Llama 3.2 3B model that operates entirely offline on a single computer with 8 GB of VRAM, and the evaluation of its response time, speed, and memory use.

2. By utilizing QLoRA-based behavioral fine-tuning, the research observes how the modifications to the model occur, such as the adjustments in its response type and format, citation method, and treatment of questions not addressed in the manuals. The model's backers measured the veracity of its responses, compliance with instructions, and approach to queries lacking backing and conducted comparison of the newly configured model with the same 4-bit model that lacks the adapter. The fine-tuning does not add new facts to the model.

3. The study includes the construction of a knowledge base from the handbooks and manuals of University of Cabuyao, in which the document title, issuing office, year, and page of each passage are recorded so that every answer can be cited.

4. The system retrieves relevant passages from the knowledge base through a Retrieval-Augmented Generation pipeline and generates answers with citations; its retrieval accuracy, groundedness, and citation accuracy are evaluated.

5. The system decides, for each query, between a cited answer, a referral to the responsible office, and an out-of-scope message, and its routing accuracy is evaluated.

6. The prototype's performance is evaluated on its functional suitability, performance efficiency, and reliability using selected characteristics of ISO/IEC 25010:2023, with approximately 80 English test queries stratified into answerable, in-scope-without-evidence, and out-of-scope cases.

7. During the Academic Year 2026-2027, the study is carried out between the prototyping, the knowledge base construction, the behavioral fine-tuning, and the evaluation of the system. The offline requirement mentioned previously concerns solely the prototype in use. The LLM judge that is employed to evaluate the correctness, groundedness and reference accuracy of answers taken during the process of evaluation is an extra tool that is not used in the system.

Limitations

1. Each inquiry made by users will be assessed individually and no previous knowledge will be utilized. The information that the system provides is only based on the manuals and guides of University of Cabuyao.

2. The system provides answers to the questions that can be answered by the handbooks or guides. Questions that demand a personal opinion or demand looking at the student's records, such as grades or balance in bank account, will be sent to a department responsible for such requests.

3. The system will use the university manuals shown in Table 9 which are certified to be the latest approved versions as of September 2026.

4. The prototype will be tested regarding about 80 questions. The results demonstrate only performance of the prototype during testing and are not claimed to be statistically significant.

5. The prototype operates only on the local machine where it is installed and is not deployed as an online service, limiting access to on-site use only.

6. The findings of this research are limited to Llama 3.2 3B model, the mentioned 8 GB VRAM hardware configuration, selected university guides and created English test-question database. Thus, the results may not be generalized to other language models, hardware configurations, databases, languages, and universities.

Definition of Terms

This segment includes the significant expressions and principles that are described for making sense of the ideas provided in the article. Along with that, the descriptions that are given here are supposed to get rid of any confusion and make sure the interpretations are constant throughout the text.

Abstaining. The unique approach of the model not responding when there is not enough information provided to answer the question, and which sets it apart from Query Routing, which determines the necessity of bringing up the queries in the first place.

Accuracy of Answers. The ability of the created answer to express the same information as the one prepared beforehand from the manual. It is noted that each answer has to be assessed for accuracy in accordance with the rubric used in the experiment by an LLM judge who compared this answer with his own notes.

Behavioral Fine-tuning. The kind of fine-tuning which does not affect the knowledge but the peculiarities of expression of the model's answers. In the current work, behavioral fine-tuning is responsible for teaching the model the way of responding.

BM25 (Best Matching 25). A ranking function that scores a passage by how often a query's words appear in it and how rare those words are across the whole knowledge base. It matches on the words themselves rather than on meaning, which makes it effective for exact identifiers such as policy names, course codes, and fees.

Chunking. The process of splitting long documents into smaller, manageable passages so that the retrieval system can search through them more effectively.

Citation Accuracy. A measure of whether the citation attached to an answer points to the passage that actually contains the stated information, with all four metadata fields — document title, issuing office, year, and page — matching that passage. A citation that is correctly formatted but points to the wrong document or page is counted as inaccurate.

Citation Metadata. The four descriptive fields recorded for every passage in the knowledge base — document title, issuing office, year, and page number — which together allow an answer to be traced to its institutional source.

Cited Answer. The response path taken when the knowledge base contains sufficient evidence for a query: the system answers from the retrieved passages and attaches the citation for the passage used.

Cohen's Kappa (κ). It indicates how two judges, in the current study, LLM judge and human evaluator, coincide with one another taking into account only the actual coincidence and ignorance that could appear purely due to lucky chance. The agreement score of 0 means there is no coincidence better than a coin toss, while the score of 1 means total coincidence. Here it is used to verify that the scores awarded by the LLM judge can be trusted at the further use on the whole testing set.

Dense Retrieval. A search method that looks at the actual meaning behind the student's question, allowing the system to find relevant manual passages even if the student uses different words.

Evidence Sufficiency Threshold. The pair of retrieval-score thresholds, one for lexical retrieval and one for dense retrieval, used to decide whether the retrieved evidence is sufficient to answer a query. The lexical threshold is applied to a normalized score, the top-ranked passage's BM25 score divided by the highest score the query could attain, so that questions of different lengths are judged on the same 0-to-1 scale. Evidence is treated as sufficient when either threshold is met; when neither is met, the query is passed to the scope check, which decides between an office referral and an out-of-scope message instead of proceeding to answer generation. Both thresholds are fixed before evaluation begins and are reported with the results, so routing behavior is not adjusted after the outcomes are known.

Fine-Tuning. The process of further training an already-pretrained language model on additional examples so that it performs better on a specific task. Fine-tuning may change what a model knows or how it responds; this study uses only the second kind, described under Behavioral Fine-Tuning.

Generation Speed. The rate at which the model produces its answer, measured in tokens per second and computed as the number of output tokens divided by the generation time alone. Unlike response time, it excludes retrieval and routing, so it describes the model's output rate rather than the system's total wait.

Gold Passage. The passage in the knowledge base identified by the researchers, before evaluation begins, as the one containing the answer to a given test query. It serves as the reference against which retrieval accuracy is scored.

Groundedness. A measure of how far a generated answer stays within the evidence it was given. Each factual claim in an answer is checked against the passages retrieved for that query, and the answer is counted as grounded only when

every claim it makes is supported by those passages. A claim that is true in general but absent from the retrieved passages is not grounded.

Hallucination. When the language model generates an answer that sounds correct but is actually false or not supported by the official university handbooks.

Helpdesk Chatbot. An automated system designed to answer student inquiries by retrieving information from official university manuals and generating responses with proper citations.

Hybrid Retrieval. A search pipeline that combines both exact keyword matching (BM25) and meaning-based search (Dense Retrieval) to get the best possible information for the generator.

Instruction Following. A measure of whether a response obeys the output rules given in the system prompt — answering directly, using the required response format, and attaching a citation with document title, issuing office, year, and page. It is scored against a fixed format checklist and is judged separately from answer correctness, so that a factually right answer given in the wrong format still counts as an instruction-following failure.

ISO/IEC 25010:2023. A recognized software quality model used in this study to evaluate the prototype's functional suitability, performance efficiency, and reliability.

Knowledge Base. The system is a searchable repository of official information about the students of the University of Cabuyao and the records of the university.

Llama 3.2 3B Instruct. It is a model released by Meta containing 3 billion parameters, which is the instruction-tuned version of the model that has been trained to comply with written directions.

LoRA (Low-Rank Adaptation). A fine-tuning method that leaves the model's original weights frozen and trains a small set of additional low-rank matrices, called adapters, alongside them. Because only these adapters are trained, far less memory is required than full fine-tuning, and the original model remains unchanged.

Memory Use. The peak amount of graphics memory (VRAM), in gigabytes, is occupied by the quantized model and its adapter while an answer is being generated, recorded during the test run rather than at idle. System RAM use is reported alongside it. The prototype is considered to satisfy its hardware constraint when peak VRAM use stays within the 8 GB limit of the test machine.

Offline Deployment. Running the entire chatbot system locally on a single computer without needing internet access or external cloud servers.

Out-of-Scope Message. The response path taken for a query outside university helpdesk concerns entirely: the system states that the question falls outside what it covers and does not attempt an answer.

Passage. A single searchable unit of text produced by chunking a source manual, and the smallest unit the system stores, retrieves, and cites. Each passage carries its own citation metadata, so that any statement drawn from it can be traced back to one page of one document.

Query Routing. The mechanism that looks at a student's question and decides whether to give a cited answer, refer them to an office, or send an out-of-scope message.

Quantization. The process of storing a model's weights at a lower numerical precision than they were trained at, reducing the memory the model occupies. In this

study, 4-bit quantization has been employed so that the model with Llama 3.2 3B can fit on the test machine with an 8 GB VRAM. The study does not perform comparisons between the 4-bit model and variants of the model with better precision since the quantization is kept constant for all test conditions and its effect on output quality is derived from earlier studies [14].

QLoRA (Quantized Low-Rank Adaptation). A fine-tuning method that combines quantization and LoRA, allowing a compressed model to be fine-tuned using limited hardware. In this study, QLoRA is used to apply behavioral fine-tuning to the 4-bit quantized Llama 3.2 3B model.

Reciprocal Rank Fusion (RRF). The step in the retrieval pipeline that merges the search results from both BM25 and dense retrieval into one final ranked list.

Response Time. The total elapsed time, in seconds, from the moment a student submits a query to the moment the complete answer and its citation are displayed. It covers retrieval, routing, and generation together, and is measured on the designated 8 GB VRAM test machine with the model and index already loaded into memory. It is reported separately for each routing outcome, cited answer, office referral, and out-of-scope, as the median and the slowest 95th percentile within each route.

Retrieval Accuracy. A measure of whether the retrieval pipeline finds the right evidence. For each answerable test query, retrieval is counted as successful when the gold passage appears among the top five passages returned by hybrid retrieval and the measure is reported as the percentage of such queries where this occurs. It describes the retriever only, and is scored independently of whether the generated answer is correct.

Retrieval-Augmented Generation (RAG). The main architecture of the system where the chatbot first retrieves relevant text from the official manuals and then uses that exact text to generate its answer.

Routing Accuracy. A measure of whether each query is sent down the response path it should take. Every test query is assigned an expected route before evaluation, and routing accuracy is the percentage of queries the system assigns to that same route. Results are reported as a three-by-three confusion matrix across cited answers, office referral, and out-of-scope, so that the direction of each routing error is visible.

Small Language Model (SLM). A language model with a parameter count low enough to run on a single consumer graphics card rather than on server hardware, commonly taken to mean models of roughly one to eight billion parameters. There is no formal cutoff; the term is used in this study to mean models deployable within the 8 GB VRAM limit, of which the 3-billion-parameter Llama 3.2 3B is the one used here.

Stratum (plural: Strata). A subgroup the 80-query evaluation set is divided into before evaluation begins, so that each of the system's three response paths is deliberately represented rather than left to chance. This study uses three strata, defined against the knowledge base itself and listed in Table 11: Answerable (≈40 queries, settled by a passage in the manuals), In-Scope Without Evidence (≈25 queries, within the manuals' subject matter but not settled by any passage), and Out of Scope (≈15 queries, outside the manuals' subject matter entirely). Sampling separately within each stratum ensures that a route which occurs less often in practice, such as out-of-scope questions, is not accidentally underrepresented or absent from the evaluation set.

Unsupported Query. A question the knowledge base cannot answer, either because no passage in the approved manuals contains the information, or because answering would require a student's personal records or the judgment of university staff. A query may be university-related and still be unsupported.

VRAM (Video Random Access Memory). The memory on a computer's graphics card. This study is constrained to an 8 GB VRAM limit, which dictates the hardware setup for the prototype.

CHAPTER II

REVIEW OF RELATED LITERATURE AND STUDIES

Reviewed Literatures

Quantization and Offline Deployment of Small Language Models

The work of Chen et al. [14] regards post-training quantization (PTQ) as a geometrical problem and reveals that 4-bit quantization on Meta's Llama-3.2-3B-Instruct essentially tracks the uncompressed 16-bit baseline, thus showing impressive congruency with respect to performance. Their geometrical analysis provides the theoretical framework and the empirical basis necessary for Objective 1 and validates the use of 4-bit quantization for the purpose of reducing the size of the model for offline implementation without negative implications for language quality.

Liu et al. [15] present ParetoQ, a unified framework comparing quantization at 1, 1.58, 2, 3, and 4 bits, and identify a learning transition between 2 and 3 bits: models quantized to 3 bits or above retain distributions close to the original pre-trained weights, whereas those at 2 bits or below undergo drastic representational change. Their GPU-latency measurements on the LLaMA 3.2 family (1B, 3B, 8B) further show that at the 3B scale, reducing from 4-bit (W4A16) to 2-bit yields negligible speed-up because kernel-conversion overhead dominates, making 4-bit the more favorable speed–accuracy operating point. This finding corroborates the quantization target adopted in Objective 1. The researchers note, however, that the study's broader conclusion favours sub-4-bit quantization at larger model sizes and that its latency figures were measured on datacenter-class hardware;

establishing the corresponding response-time and memory characteristics on an 8 GB consumer GPU is a distinct contribution of the present study.

QLoRA and Parameter-Efficient Fine-Tuning

Dettmers et al. [8] introduce QLoRA, a parameter-efficient fine-tuning method that combines 4-bit NormalFloat (NF4) quantization, double quantization, and paged optimizers to reduce memory usage to a fraction of that required by full-precision training while preserving 16-bit task performance. QLoRA serves as the foundational method for Objective 3, which applies it to fine-tune a 4-bit Llama 3.2 3B model within an 8 GB VRAM budget.

Complementing the memory-efficiency case, Zhou et al. [17] show that fine-tuning with as few as 1,000 high-quality examples—without reinforcement learning—suffices to produce coherent, well-formatted responses, demonstrating that instruction-tuning aligns surface behaviour rather than injecting new factual knowledge. This finding supports the data-efficient approach adopted in this study, in which a curated set of helpdesk examples teaches the model proper formatting, citation placement, and abstention conventions.

Empirical feasibility is confirmed by Rauf et al. [18], who fine-tune 4-bit Llama 3.2-3B alongside Gemma 3 and Qwen 3 for multilingual spam detection on resource-constrained hardware, reducing VRAM consumption from 6.43 GB to 2.24 GB while improving task accuracy from 47 % to 70 % (F1: 0.45 → 0.69). Because their downstream task is classification rather than open-ended question answering, the researchers treat this result as evidence of memory-budget feasibility only, not as a predictor of answer quality in a helpdesk setting.

To measure whether the fine-tuned model actually follows the behavioral rules it has been taught, the researchers draw on IFEval [19], a benchmark that objectively scores compliance with verifiable instructions—such as formatting constraints—rather than relying on subjective human or LLM-as-judge evaluations. IFEval's design maps directly onto the helpdesk requirements of Objective 3, where adherence to prescribed formatting, citation inclusion, and abstention conventions must be demonstrably measured.

Knowledge Base Construction and Chunking

The method “semantic layout chunking” is introduced by Qin et al. [20] , which uses visual cues such as headings, tables, and section dividers to produce chunks that are semantically coherent and thus alleviate the issue of fragmentation present in RAG systems due to random splitting of sections. The technique provides evidence for the chunking method used in Objective 3 which entails the segmentation of student manuals based on structure and the annotation of each passage with document-title, issuing-office, year, and page data.

Recent research on chunking applied in RAG has shown that the segmentation decision should be based on the semantics of the text rather than arbitrary number of tokens in various segments. Qin et al. [20] utilize the cues from visual layout, i.e head lines, tables, and separator signs to create the chunks; Pham and Phuong [21] implement that concept hierarchically whereas they retain connections between the segments and their headings to ensure that all chunks preserve their complete context; and Kiss et al. [22] provide a formal model of boundary identification called "Max–Min semantic chunking", an algorithm that can identify the segmentation points of minimal similarity between segments giving the scores of AMI of 0.85–0.90 according to various tests. All these methods aid in

producing the chunking strategy used in Objective 2, as the content is split based on the structure per [20] and [21], the positions are checked for coherency in semantic meaning per [22], and information concerning the title, issuing office, year, and page number is added to each segment for future citation.

Hybrid Retrieval and Retrieval-Augmented Generation

In their study, Cormack et al. [23] established that Reciprocal Rank Fusion (RRF) consistently outperforms both individual rankers as well as Condorcet-based combination methods when merging heterogeneous retrieval lists. This observation is a direct support for the retrieval design of Objective 4, which relies on RRF to combine the results of keyword-based BM25 retrieval and of dense semantics in a unified modified context window.

Wang et al. [24] investigated both sparse and dense retrieval within RAG echnologies utilizing limited hardware and concluded that using complementary strategies allows obtaining the maximum recall with the least computational effort. This supports the presence of a dual-retriever approach in Objective 4, which utilizes BM25 and neural embedding to meet the challenge of both the specific applicability of the term and the general nature of students' queries.

Nevertheless, even if the required passage is retrieved, there is no guarantee that the model will be able to quote it correctly. Van Dort and Heuss [25] demonstrate that citation behavior can be attributed to a distributed 'attributional ensemble' of attention heads and MLP layers rather than to a standalone citation mechanism, pointing out that the reasoning of the model may be inconsistent with its calculations. This serves as a basis for the citation verification protocol applied in this research, as the mere presence of a citation does not prove that the cited passage was used.

Routing, Abstention, and Out-of-Scope Handling

Zhang and Wu [26] propose an evidence-sufficiency benchmark that grades retrieved context along a five-level gradient—from fully sufficient to absent or conflicting—and measures how often each of seven models (across five families) answers when it should abstain. At the most degraded evidence level, over-answer rates range from 65 % to 91 %, demonstrating that reliable abstention does not emerge from the language model alone. This finding directly motivates the routing logic of Objective 5, which evaluates passage adequacy before generation rather than delegating the sufficiency judgement to the model itself.

Liu et al. [27] tackle the complementary issue of coming up with test queries for which the relevant corpus lacks sufficient proof. Their CRUMQs pipeline extracts themes from the target corpus, gets recent papers on that subject from external repositories, and produces multi-hop queries whose answers are based on information that is missing or only partially present. The verifications show the top scoring passages in the corpus cannot answer any of those queries. This verification procedure provides a directly adaptable template for constructing insufficient evidence test queries from the student manuals under Objective 5. Because CRUMQs queries are derived from corpus-internal topics, they probe evidence sufficiency rather than domain membership; the out-of-scope routing decision is therefore evaluated separately.

Review of Studies

Institutional Adoption, Domain Context and Dataset Constraints

Universities are increasingly integrating chatbots for frequently asked questions related to students admission, certification, academic requirements and institutional procedures while reducing repetitive staff workload [28, 30]. However, the effectiveness of chatbots depends on how accurately the knowledge base understands the students request. For instance, Peyton and Unnikrishnan [28] were able to achieve an F1 score of 0.99 by using Sentence-BERT on a structured data from the university FAQ dataset while Bilquise [29], bilingual chatbot scored 80% and 75% in English and Arabic respectively missed probably 20% of possible questions due to insufficient data. Essop [30] states that a university chatbot should not only work effectively but should also be useful for providing information to students requests. These findings support the proposed systems use of verified University of Cabuyao documents, controlled document preprocessing, and regular review of unsupported inquiries to identify gaps in the knowledge base.

The studies reviewed indicate that utilizing the specific information about the institution rather than only general data is an efficient strategy [28]-[30]. They also point out from the experience that a bot should not provide answers if there is not enough information in the database. Accordingly, the proposed helpdesk will retrieve from approved university documents and will provide an office referral or out of scope response when adequate evidence is unavailable. These approaches are expected to improve the relevance of responses while reducing unsupported institutional claims.

Architectural Transitions: RAG, Grounding, and Reliability Mechanisms

To address the limitation of fixed FAQs and intent matching, new systems utilize the large language models and Retrieval-Augmented Generation (RAG) techniques, which retrieve certain relevant segments of text before crafting the response. This technique works well for academic advising as many queries regarding missing information about university rules, rights, processes, and services require certain evidence specific for a certain institution [29] - [31]. For, instance, the application of the AI powered chatbot for academic advising was represented in the research carried out by Tamascelli et al.[31] While [35] showed that the chatbot based on LLM was able to provide the answers based on the information from the academic courses with 88% accuracy. These studies support the proposed system's use of RAG over official university documents and suggest that the knowledge base can be updated as approved policies and procedures change.

Although RAG can considerably reduce the risk of hallucination, it does not eliminate errors in retrieval or generation [32], [34], [35]. The use of proprietary RAG systems for the evaluation of legal artificial intelligence tools proved that even in critical situations, hallucination may occur from 17% to 33% of cases [34]. Similar tendency can be observed in the university systems, as in the outcome of the evaluation of the work of MoodleBOt, its accuracy was 88% and it turned out to be effectively used in the course materials despite the issues experienced by the system while finding errors in the responses [35].

Reliability means that the system must refrain from making decisions in cases of insufficient evidence and that citations have to support the received answer. [33] Shown that specific models can be trained to refrain from responding where they are uncertain, while [36] identified abstention as an important safeguard against

unsupported responses. Some issues have also been raised by van Dort and Heuss [25] regarding false sense of security created by citations that do not serve as support for generated statements. These approaches support the proposed system's QLoRA-based behavioral adaptation, evidence-based answering, office referral, out-of-scope handling, and preservation of document title, issuing office, year, and page number. They also justify evaluating abstention and citation accuracy independently rather than treating a fluent, cited response as automatically reliable.

Standardized Software Quality Evaluation and Local Research Gaps

Previous studies demonstrate that ISO/IEC 25010 can be used to evaluate institutional information systems, educational technologies, and conversational applications. Lagman et al. [38] applied the framework to a Philippine State University and College system, while Kwok, Chen, and Ramos [39] used it to assess an AI-supported university application. Barletta et al. [40] also showed that ISO/IEC 25010 can be used for evaluating chatbots and Rojas et al. [41] confirmed its application in different areas of research. These studies provide support for evaluating the proposed chatbot as a complete software system rather than measuring only its response accuracy. In this study, the parameters of functional suitability, performance efficiency, and reliability will be measure to check whether the system can fulfill its functions of a helpdesk and provides effective service.

In a study at the Polytechnic University of the Philippines and Centro Escolar University-Manila, virtual assistant services for learners have been created and evaluated using ISO/IEC 25010 [42], [43]. A separate local research studies on a chatbot AI belong to an online shop in Nueva Ecija have used the Expectation Confirmation Theory and the ISO/IEC 25010 criteria, showing that the standard for chatbot evaluation is already being applied and not necessarily limited to institutions

of higher education [44]. Therefore, these studies provide support for the feasibility of AI services and convey approaches for assessing the proposed system through software quality and user oriented criteria.

In addition, most studies that have been reviewed in the current study use a Large Language Models (LLMs), which typically comprise billions of parameters and necessitate powerful computing devices, like in the case of Llama 2 models containing between 7 and 70 billion parameters, as noted in [48]. The current research is carried out with regards to the Small Language Model (SLM), which is a type of model that is designed to be lightweight and resource-efficient, and which is potentially suitable for offline usage. The Phi-3 technical report by Abdin et al. [48] shows that a small model with only 3.8 billion parameters can provide proper performance in languages and be used locally on computers with limited resources [48]. This is an important difference because the system that is proposed in this paper is supposed to be designed for the needs of the University of Cabuyao and thus should work using only the available hardware of the institution.

Overall, the reviewed studies provide support for both the development and evaluation of the proposed system. According to their findings, the reliance on a verified domain-specific corpus, retrieval from authoritative documents, testing retrieval and generation process, controlling for citation accuracy, and abstention in case of insufficient evidence should be adopted [28], [29], [31-37]. Previous findings related to ISO/IEC 25010 further confirm the significance of assessing the functional suitability, performance, and reliability of the developed helpdesk [38-42]. By using these recommendations, the proposed desk is aimed at fulfilling the specific information needs of the institution while respecting its constraints.

Patent Search

The invention [45] discloses a system in which a user provides a question to be answered from detailed, dense, or otherwise complex documents. The processing system converts the question to a structured query language query and generates an embedding from the question, augmented by temporal data, synopses, themes, or other relevant information. The embedding is compared to embeddings generated from documents of a knowledge base to identify documents that are relevant to the question, and to rank such documents for their relevance. Highly ranking documents are combined with the query and provided to a language model that returns an answer to the question. A source for the answer is identified in at least one of the documents, and the answer together with the identified documents is presented to the user. The claimed system further recites extracting text from the documents, partitioning the extracted text into text chunks, augmenting the chunks with metadata, and indexing them in a searchable database, where the language model was trained on a corpus that does not include those documents. [45]

The similarities of the invention [45] to the current research are the following:

1. Both construct a knowledge base by extracting text from source documents, partitioning it into chunks, attaching metadata, and indexing the chunks for retrieval.

2. Both convert the user's question into an embedding and compare it against document embeddings to select the passages most relevant to the question.

3. Both supply the selected passages together with the question to a language model, which generates the answer from those passages.

4. Both identify the source of the answer and present it to the user, so that the response can be traced back to a document.

5. Both keep the factual content in the indexed documents rather than in the model's trained weights, so that the model is not the source of the facts.

The differences between the invention [45] and the current research are the following:

1. The invention [45] is claimed as a system operating on a provider network, that is, a cloud service; the current research operates entirely offline on a single computer with 8 GB of VRAM and no internet connection.

2. The invention [45] retrieves using embedding similarity alone; the current research uses hybrid retrieval, combining BM25 lexical search with dense embedding search fused through Reciprocal Rank Fusion, because form codes and acronyms in the manuals favor lexical matching while student phrasing favors semantic matching.

3. The invention [45] identifies the source document; the current research records the document title, issuing office, year, and page of every passage, and the application independently validates in code that each cited identifier belongs to evidence actually supplied to the model before the answer is displayed.

4. The invention [45] returns an answer to the question; the current research decides among three outcomes for each query — a cited answer, a referral to the responsible office, or an out-of-scope message.

5. The invention [45] employs a general large language model; the current research employs a 4-bit quantized small language model with a QLoRA adapter that adapts answering behavior only and adds no facts to the model.

The invention [46] addresses the problem that current chatbot systems cannot understand enough different topics to converse with students who have questions about college admissions, financial aid, courses, and other topics on the path to and through college. It further observes that such chatbots have difficulty understanding misspellings, slang, and context-specific language of the kind students use, that learning new topics is time-consuming, and that it is difficult for administrators to participate in student conversations. To address these problems, the inventive chatbot uses a natural language processor, for example a neural network, to receive, classify, and respond to queries on thousands of different topics. The chatbot can also request real-time assistance from an administrator when faced with a difficult query, and learn on-the-fly from the administrator's response. [46]

The similarities of the invention [46**] to the current research are the following:**

1. Both are text-based conversational systems built specifically to answer the questions of university students about institutional matters such as admissions, enrollment, and academic policies.

2. Both aim to reduce the load of repetitive student inquiries that would otherwise be handled by university staff.

3. Both use a trained language model to interpret the student's query rather than requiring the student to select from a fixed menu.

4. Both accept that some queries cannot be answered by the automated system and must be directed to a person.

The differences between the invention [46] and the current research are the following:

1. The invention [46] classifies queries into topics using a natural language processor and answers from curated content; the current research retrieves passages from the university's official manuals at query time and generates the answer from that retrieved text.

2. The invention [46] provides no citation for its responses; the current research cites the document title, issuing office, year, and page for every answer it gives, and verifies the citation in code before display.

3. The invention [46] requests real-time assistance from a human administrator and learns on-the-fly from that reply; the current research has no human in the loop, and no conversation alters what the system knows — updating its information requires re-indexing the source documents.

4. The invention [46] operates as a hosted service with a load balancer serving many concurrent users; the current research runs offline on a single computer with 8 GB of VRAM, tested with one user at a time.

5. The invention [46] predates retrieval-augmented generation; the current research applies a 4-bit quantized small language model with QLoRA behavioral fine-tuning and hybrid retrieval, technologies that were not available when the invention was filed in 2017.

Research Gaps

While Retrieval-Augmented Generation (RAG) and conversational agents are increasingly being explored for educational and university helpdesks [2, 4], existing studies often examine quantization, QLoRA-based fine-tuning, and document retrieval as separate approaches [8, 9]. Many existing systems also rely on online or cloud-based services, which may not be suitable for environments with limited computing resources or unreliable internet access [1, 2]. Even though the previous studies have shown the potential of smaller language models and RAG use in academic information services, there is still limited research on using them together in a fully offline university helpdesk system.

A research gap therefore remains in evaluating how 4-bit quantization, QLoRA-based behavioral fine-tuning, and RAG can work together in a locally deployed 3B-parameter Small Language Model for a university helpdesk. In particular, further study is needed on the system's performance under limited hardware, answer and citation accuracy, handling of unsupported questions, and appropriate routing of different types of user queries. Furthermore, limited comparative evidence is available regarding which configuration provides the best balance between answer quality and resource efficiency under the specified hardware constraint. Specifically:

1. **Offline Deployment on Limited Hardware.** Previous studies show that 4-bit quantization can reduce memory use while maintaining acceptable model performance [14, 15]. However, there is still limited research on the actual performance of small language models when deployed fully offline on consumer-grade hardware with limited VRAM. In particular, response time, speed, and memory use under an 8 GB VRAM constraint need further

evaluation. This study addresses this gap by deploying Llama 3.2 3B in 4-bit quantized form and evaluating its offline performance.

2. **QLoRA for Helpdesk-Specific Behavior.** QLoRA allows language models to be fine-tuned with lower memory requirements [16, 18], while previous studies show that small datasets can improve instruction-following behavior [17]. However, the role of the QLoRA technique in the context of university helpdesk behavior, e.g., following a required response format, providing citations, and properly handling unsupported questions, has not been studied extensively. This study aims to address this gap by adapting Llama 3.2 3B to the helpdesk behavior using QLoRA and comparing the adapted model to the unadapted 4-bit version in order to evaluate the impact of adaptation.

3. **Citation-Aware Knowledge Base.** RAG studies show that document structure and chunking can affect retrieval quality [20–22]. However, retrieving relevant information does not always ensure that generated citations correctly support the answer [25]. There is limited research on citation-aware knowledge bases for university helpdesks that connect each passage with source details such as document title, issuing office, year, and page. This study addresses this gap by creating a structured knowledge base that connects each passage with source details such as document title, issuing office, year, and page.

4. **Evidence-Based Query Routing.** Studies have shown that RAG systems may still generate answers when the available evidence is insufficient [26, 27]. However, limited research has examined simple routing mechanisms that distinguish between questions that can be answered, questions that require an office referral, and questions that are outside the system's scope. This

study addresses this gap by implementing a query-routing mechanism and evaluating its routing accuracy.

5. **Small Language Models for University-Specific Needs.** Existing literature relies heavily on Large Language Models (LLMs) with tens to hundreds of billions of parameters that require high-end cloud systems and internet connections. In contrast, Small Language Models (SLMs) are designed to be lightweight, resource-efficient, and capable of local execution. However, there are very few studies explores lightweight SLMs tailored specifically to meet the internal policy needs and offline requirements of university help desks. To fill this gap, this research employs a Small Language Model developed specifically for the privacy and documentation needs of a university.

Synthesis

Previous research has investigated the application of Retrieval-Augmented Generation and Small Language Models for automating university helpdesk systems, demonstrating advantages such as reduced hallucination, evidence-based responses, and lower computational needs compared to large, cloud-based models. However, most existing studies treat quantization, fine-tuning, and retrieval as separate techniques rather than a combined system, and there are still challenges such as maintaining citation accuracy, ensuring the model properly refuses to answer when evidence is insufficient, and running reliably offline on limited hardware. Recent developments in model compression and parameter-efficient fine-tuning, in particular 4-bit quantization and QLoRA, have shown potential in overcoming these challenges by allowing language models to run efficiently on resource-constrained devices without a substantial drop in performance. This research aims to address the gap in the literature by combining 4-bit quantization,

QLoRA-based behavioral fine-tuning, and hybrid RAG to develop a robust and reliable offline university helpdesk chatbot capable of running efficiently on a single computer with only 8 GB of VRAM. This study aims to help improve the locally deployed, resource-efficient chatbot technology for use in Philippine universities using University of Cabuyao as a case study by addressing issues with citation accuracy, evidence-based query routing, and appropriate handling of unsupported questions and evaluating the prototype with ISO/IEC 25010:2023 standards.

CHAPTER III

RESEARCH METHODOLOGY

This chapter presents the methodology and technical framework used in the development and evaluation of the offline university helpdesk chatbot for University of Cabuyao. It presents the systematic procedures that were utilized to develop, optimize, operate, and assess the prototype that uses a Small Language Model with a 4-bit quantization integrated with Quantized Low-Rank Adaptation (QLoRA) and Retrieval-Augmented Generation (RAG). This section explains the structure of the system architecture, hardware environment, the software stack, operational workflows, and evaluation protocols designed to test whether the system delivers accurate, evidence-grounded, and verifiable cited responses entirely offline within an 8 GB VRAM constraint.

Project/Research Design

This section presents the system architecture of the offline university helpdesk prototype developed in the study entitled “Optimizing Local Small Language Models Using 4-Bit Quantization, QLoRA, and RAG for University Helpdesks.” It illustrates how the system's hardware and software components operate and how they interact with one another to provide accurate, evidence-grounded, and citation-backed responses to student inquiries using official university documents within the setting of University of Cabuyao.

This study follows the Design Science Research Methodology [50], an applied developmental design in which the research output is an engineered artifact: the offline university helpdesk prototype. Its contribution is twofold. The first is the

artifact. The language model, the fine-tuning method, and the retrieval algorithms are established and adopted from prior work; the researchers develop the components that make them function as a citation-verified helpdesk on a single 8 GB consumer graphics card: a knowledge base built from the university's manuals with a four-field citation record for every passage, a behavioral training set used to fine-tune how the model answers, cites, and declines, a two-stage query router that decides each response path from retrieval evidence before the model is called, and a program that verifies each citation before an answer is displayed. The second is evidence: a quantitative evaluation of that artifact against criteria fixed before testing and organized by ISO/IEC 25010:2023, establishing whether this configuration meets those criteria.

During system development, two controlled configuration comparisons will be conducted. First, the unadapted and QLoRA-adapted versions of the same 4-bit Llama 3.2 3B model will be compared under identical prompts, evidence contexts, generation settings, hardware conditions, and held-out comparison questions. The comparison will determine whether QLoRA improves answer correctness, instruction following, and unsupported-query handling.

Second, BM25-only, dense-only, and RRF-fused hybrid retrieval configurations will be compared using the same development queries and retrieval-accuracy measure. These comparisons are configuration-selection activities conducted before final evaluation.

After the model and retrieval configurations have been selected, the settings will be frozen. Only the finalized complete system will then be evaluated using the separate 80-query evaluation dataset and the selected ISO/IEC 25010:2023 measures.

The prototype is intended for use by students of University of Cabuyao seeking information from the university's official handbooks and manuals, within the institutional setting described in Chapter I.

System Architecture

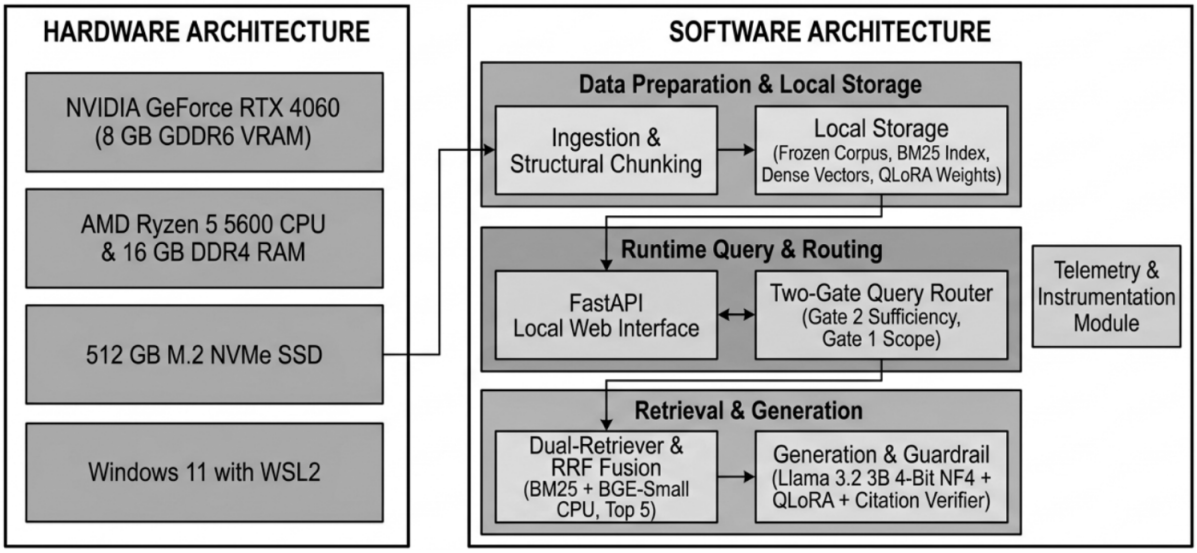


Figure 2. System Architecture of the Offline University Helpdesk

Figure 2 splits the architecture into two halves. The left side shows the hardware — an NVIDIA RTX 4060 (8 GB VRAM), an AMD Ryzen 5 5600 with 16 GB of DDR4 RAM, a 512 GB NVMe SSD, and Windows 11 with WSL2. The right side is the software stack that runs on top of it. Every software decision traces back to a hardware limit, most of all the 8 GB VRAM ceiling on the RTX 4060, which forced the selection of a 3-billion-parameter model, 4-bit quantization, and CPU-bound embedding inference.

The software side stacks three layers. Data Preparation & Local Storage runs once before evaluation: the three manuals are cleaned, chunked, tagged with citation metadata, and stored alongside the BM25 index, dense embeddings, and QLoRA weights — all on the local SSD with no internet dependency. Runtime Query & Routing handles live queries: a student's question enters through a localhost FastAPI server, gets scored by both retrievers, and reaches the two-gate query router. The router checks evidence sufficiency first — if scores are strong enough, the query proceeds directly to answer generation without ever triggering the scope classifier. Only when evidence is insufficient does the second gate decide between an office referral and an out-of-scope decline.

In the Retrieval & Generation layer, BM25 and BGE-Small each produce a ranked list, Reciprocal Rank Fusion merges them, and the top five passages are sent to the quantized Llama 3.2 3B model with its QLoRA adapter. The model generates a cited answer, and a citation verifier checks every field against the retrieved passages before display — any mismatch suppresses the response. A Telemetry & Instrumentation Module logs latency, token throughput, and peak VRAM for every query throughout the evaluation.

Hardware Architecture

The prototype was built and evaluated on a single, self-contained workstation to satisfy the offline and resource-constrained requirements of the study. No cloud services, remote GPUs, or external servers were used at any stage — all processing, from fine-tuning the adapter to running the final evaluation queries, took place on the same local machine. This setup reflects the practical reality the study targets: a university department that needs to deploy a help desk without relying on

paid API subscriptions or stable internet access. The workstation was assembled around a consumer-grade GPU with 8 GB of VRAM, which acts as the binding hardware constraint for the entire software design. Table 1 summarizes each hardware component, its specifications, and its assigned function within the system.

Table 1 lists the hardware used in the prototype, along with specifications and what each component is responsible for during operation.

Table 1

System Hardware Specifications and Roles

Hardware Component	Specifications	Functional Role
Dedicated GPU	NVIDIA GeForce RTX 4060, 8 GB GDDR6 VRAM	Runs the 4-bit quantized Llama 3.2 3B model during both inference and fine-tuning. Holds the base weights, QLoRA adapter layers, and the KV attention cache in video memory.
CPU	AMD Ryzen 5 5600, 6 cores / 12 threads	Handles everything that is not language model generation: document parsing, BM25 scoring, dense embedding computation via BGE-Small, query routing, citation verification, and the FastAPI server.

System RAM	16 GB DDR4-3200	Keeps the OS, the full chunk store, the precomputed dense embedding matrix, and the BM25 index in memory. Running embeddings from system RAM is what keeps them off the GPU.
Storage	512 GB M.2 NVMe SSD	Stores all project assets locally — manuals, parsed passages, model weights, adapter checkpoints, embedding arrays, scope exemplars, evaluation logs — so the system never needs a network connection.
Operating Environment	Windows 11 with WSL2 (Ubuntu)	Gives the CUDA-dependent Python libraries (bitsandbytes, transformers, peft) a native Linux environment while keeping the Windows host for display and file access.

The 8 GB VRAM capacity of the RTX 4060 is the single constraint from which most other hardware and model decisions follow. A 3-billion-parameter model was selected because, after 4-bit quantization, it fits within that budget alongside the key-value cache that accumulates during generation — larger models exceed the available memory under the same conditions. The dense embedding model was assigned to the CPU for the same reason: co-locating it with the language model on the GPU would exceed the VRAM ceiling. It should be noted that the Ryzen 5 5600 lacks integrated graphics, which means the RTX 4060 also serves as the display adapter. A portion of its 8 GB is therefore reserved by the Windows display compositor at all times, and the peak VRAM figures reported in the evaluation were

measured with the display attached rather than in a headless configuration. The NVMe SSD primarily affects startup performance. Loading the model weights into VRAM is the most demanding disk operation in a given session; once the weights are resident, subsequent queries are served entirely from memory.

Software Architecture

The software architecture of the offline university helpdesk chatbot comprises several modular tools, libraries, and processing layers working in coordination to ensure accurate, evidence-grounded, and resource-efficient query handling. It encompasses core backend pipeline services, local data storage structures, language model generation and guardrail routines, performance instrumentation, and a local browser-based user interface, all operating strictly without external API dependencies.

Table 2 lists the software components that make up the helpdesk pipeline, the specific technologies or methods behind each one, and what role each plays in the system.

Table 2

Software Components, Technologies, and Functional Roles of the Helpdesk Pipeline

Software Component	Technology / Implementation	Functional Role
Text Ingestion & Normalization	Python-based PDF text extraction	Extracts operative text from the three institutional manuals and removes non-actionable content such as administrative rosters, cover matter, and ceremonial messages.

Structural Chunking & Metadata Stamping	Rule-based segmentation on chapter/article/section headings	Partitions cleaned text into passages aligned to document structure and assigns four provenance fields: document title, issuing office, publication year, and page number.
Sparse Lexical Retriever	BM25 via rank-bm25, with tokenization and stemming	Ranks passages by term-frequency overlap, targeting exact matches on policy codes, acronyms, and institutional terminology.
Dense Semantic Retriever	BGE-Small-en-v1.5 (384-dim vectors), runs on CPU	Projects queries and passages into a shared vector space via cosine similarity, capturing informally phrased inquiries that lack exact keyword overlap.
Reciprocal Rank Fusion	Rank-based score merging	Merges the BM25 and dense ranked lists into a single ranking using reciprocal rank positions and selects the top 5 passages as evidence.
Two-Gate Query Router	Sequential threshold classifier	Gate 1 evaluates evidence sufficiency using normalized BM25 and cosine scores against fixed thresholds. Gate 2 activates only on insufficiency, classifying the query as in-scope (office referral) or out-of-scope (static decline) via a 1-nearest-neighbor cosine classifier.
Language Model	Llama 3.2 3B Instruct, 4-bit NormalFloat via bitsandbytes	Generates cited answers from the top 5 retrieved passages under deterministic decoding (temperature = 0, fixed seed).
QLoRA Behavioral Adapter	QLoRA via peft, low-rank matrices over frozen	Adapts the model's response behavior to answer from supplied

	quantized weights	passages only, attach four-field citations, and decline when evidence is insufficient.
Citation Verifier	Deterministic string-matching	Cross-references each cited field (title, office, year, page) against the passages supplied to the model; mismatches suppress the response before display.
Telemetry Module	PyTorch CUDA memory profiler, Python	Records response latency, token throughput, peak VRAM allocation, and routing outcomes for each query during evaluation.
Application Server	FastAPI, localhost-only binding	Serves the web interface on the loopback address and retains the model and retrieval structures in memory after initial startup
Web Client	Single-page HTML/CSS/JS, all assets bundled locally	Renders the query input, cited answer cards, office referrals, and out-of-scope notices using locally served assets with no external network requests.

The system does not rely on any database engine. Given that the three source manuals total only 199 pages — roughly a third of which is front matter removed during ingestion — the resulting knowledge base holds a few hundred passages at most. At that scale, an exhaustive comparison of every stored passage against a query vector completes almost instantly on the CPU, which makes approximate nearest-neighbor libraries such as FAISS or Annoy unnecessary. Introducing one would add a software dependency and an approximation error with no measurable speed benefit. All data assets are stored as flat local files: the read-only source manuals, the chunk store with citation metadata, the serialized

BM25 index, a contiguous NumPy array of dense embeddings, versioned QLoRA adapter checkpoints, scope exemplar vectors, an office referral list, and the 80-query evaluation dataset with its gold labels. Both the base Llama 3.2 3B Instruct weights and the BGE-Small-en-v1.5 embedding model reside on disk so that no download occurs at runtime.

Separation of System Development and Evaluation

Four structural controls keep the development process separate from the evaluation protocol.

Disjoint Dataset Partitioning. The QLoRA training pairs and the 80-query evaluation set share no overlapping items. The model's behavioral compliance is tested only on queries it never encountered during fine-tuning, so strong results indicate generalization rather than memorization.

Pre-Registered Evaluation Thresholds. Every target metric was fixed and recorded before the first evaluation run. Hardware-dependent thresholds — response time, generation speed, and the router's evidence cutoffs — were calibrated on a separate tuning set also excluded from evaluation. No threshold was adjusted after results were observed.

Deterministic and Frozen Execution. A fixed random seed, zero temperature, frozen decoding parameters, frozen chunking, and a frozen corpus ensure that re-running the evaluation yields identical outputs. The chunking procedure was locked before evaluation labeling, since the gold annotations reference specific passage identifiers that would become invalid if the corpus were re-chunked.

Function-Level Evaluation Harness. The evaluation script calls the same retrieval, routing, and generation functions that the web interface uses, bypassing the server

layer entirely. Both paths run identical code, so the interface contributes no variability to the measured results.

Project Development

The prototype will be developed through the iterative and incremental prototyping process shown in Figure 3. The system is built one component at a time, and each component is designed, built, and checked before the next one begins. The five increments correspond to Specific Objectives 1 to 5: offline 4-bit model, knowledge base, QLoRA fine-tuning, retrieval pipeline, and query router. In Objective 6, the entire system is evaluated once all increments are accepted and the settings are frozen. Each increment works under an 8 GB video memory limit and without an internet connection. All evaluation in the development phase is done on development data only; the 80-query evaluation set is only used in the final evaluation.

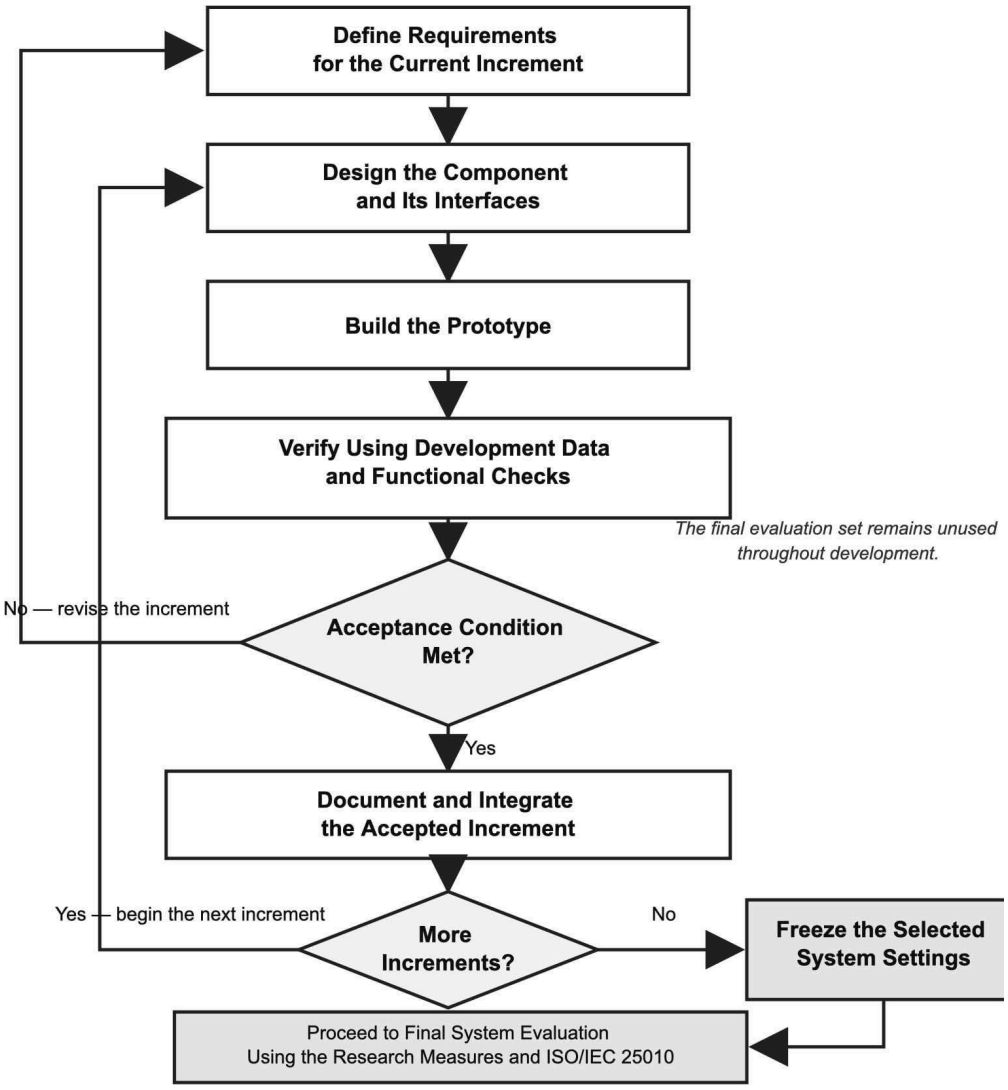


Figure 3. Iterative and Incremental Prototyping Process

Define Requirements for the Current Increment

Each stage begins by defining the required tasks and conditions for each of its parts. The requirements are based firstly on the particular goal of the increment and secondly, on the various restrictions of the project. They are stated in an unambiguous way, for example that the model must load and create with the network switched off, or that every one of the preserved passages has to contain all four citation fields. These statements later become the acceptance condition of the increment.

Design the Component and Its Interfaces

The team of researchers must now determine how the newly developed research component will fulfill its purpose and integrate itself to the existing components that have already been accepted. Interface implies that there is input received and output made by the research component. For instance, the retrieval pipeline will accept a question and produce five passages complete with citation fields. The specifications of the research component will be identified in this phase which will include quantization, router thresholds and adapter configuration, which will be set for building and verification.

Build the Prototype

The component is implemented on the development machine described in the Hardware Architecture and joined to the working prototype. Building stays within the software environment already described, and no network service is used at run time.

Verify Using Development Data and Functional Checks

The built increment is checked in two ways. Functional checks confirm that the component does what its requirements state, such as loading in four-bit form or rejecting a citation to a passage the model was not given. Checks that need questions use development data only. The development data consist of the held-out split of the behavioral training set, a development query set of 40 questions (20 answerable, 12 in scope without evidence, and 8 out of scope, in the same proportions as the evaluation set) written separately from the evaluation set and checked against it so that no question appears in both, and the scope exemplar set, which defines the scope check but is not used to judge it. These checks guide development and are not reported as results of the study; the measures and targets of the study are applied only in the final evaluation.

Acceptance Condition Met?

An increment is accepted only if all of its checks are passed. In the case that a check fails an increment is revised. The process is returned to its requirements which are re-examined in the event that the failure is due to a requirement that can not be met in the constraints of the study. The design and build steps are repeated. All failures and the revision made are recorded.

Document and Integrate the Accepted Increment

The description of the accepted increment is kept in the development log: its requirements, the design decisions and settings used, the checks done and their result, and any changes made. The component becomes a fixed part of the working prototype that the next increment builds on. After integration, the functional tests of the previous increments are redone to show that they still pass.

More Increments?

If an increment remains, development continues with it. The next increment proceeds to the design of its component, with its requirements already set by its specific objective. When the fifth increment, the query router, has been accepted, no increments remain and the process moves to freezing the system settings.

Freeze the Selected System Settings

Once all five increments are accepted, every setting chosen during development is fixed and recorded (Table 3). From this point, the same system is used in every test. If a setting has to be changed, the change is recorded and all affected tests are repeated; results taken before and after a change are not combined. The target value for each of the ten measures is also recorded before testing begins, so that the system is judged against a standard chosen beforehand rather than one chosen after the results are known.

Table 3

System Settings Fixed Before Testing

Setting	What is fixed
Retrieval settings	How the manuals are divided into passages, how many passages each search method returns, how the two sets are combined, and how many are given to the model.
Fine-tuned model	The single trained version of the model used for every test.

Setting	What is fixed
Router settings	How the system decides whether a question is within scope, and whether the retrieved passages answer it.
Response settings	How the model produces its answer and the maximum length of a response.
Targets	The value each of the ten measures must reach to be considered acceptable.

Proceed to Final System Evaluation Using the Research Measures and ISO/IEC 25010:2023

The frozen system then proceeds to final evaluation. The machine-dependent targets are set in the Preliminary Calibration Run, the 80-query evaluation set is used for the first time in Tests 1.1 to 6.3 of the Testing Procedure, and the ISO/IEC 25010:2023 evaluation is carried out as described in the Evaluation Procedure.

Development Increments

The five increments are described below. Each table states what the increment must do, how it is designed and built, how it is checked with development data, and the condition for accepting it.

Increment 1: Offline Deployment of the 4-Bit Quantized Model

The first increment is the base on which all the other components run. The language model is loaded in four bit form on the local machine, with no network access.

Table 4

Increment 1 Development Plan

Process Step	Activity in This Increment
Define Requirements	Llama 3.2 3B Instruct loads in 4-bit NF4 format and generates a complete response on the RTX 4060 within its 8 GB of video memory, with the display attached. It runs with the network disabled, with all weights stored locally. Response time, generation speed, and peak memory are logged for every query.
Design	The workstation and software environment in the Hardware and Software Architecture; quantization settings; decoding settings (temperature 0, fixed seed, maximum response length); a local FastAPI server bound to loopback with a single-page interface; and the telemetry fields to be logged.
Build	Set up Python on WSL2, store the model weights locally, load the model in NF4, and implement the local server, interface, and run logging.
Verify	With the network disabled, submit development queries through the interface. Confirm the model loads in four-bit form, returns a complete response, stays within 8 GB of video memory, and that each run logs all three fields.
Acceptance Condition	All verification checks pass.

Process Step	Activity in This Increment
Document and Integrate	A working offline generation path with recorded quantization and decoding settings, which the later increments extend.

Increment 2: Knowledge Base Construction

The second increment turns the three official manuals (Table 9) into stored, citable passages and their two search indexes, following the Knowledge Base Construction Procedure in Figure 4. Front matter that carries no student procedure is excluded from chunking, and each passage is stored with the fields in Table 10.

Table 5

Increment 2 Development Plan

Process Step	Activity in This Increment
Define Requirements	Passages come only from the three manuals, with the listed front matter removed. Every passage carries its document title, issuing office, edition year, and printed page, with the office resolved from the section's document control code. No passage exceeds the embedding model's 512-token limit. A passage missing any citation field is not admitted.
Design	Chunking along the manuals' own divisions; the passage record fields; the section-to-office lookup and office list; the lexical and dense indexes.

Process Step	Activity in This Increment
Build	Extract and clean the text, chunk it, stamp the metadata, resolve offices, and build the BM25 index and the dense embedding matrix.
Verify	Confirm that every stored passage has all four citation fields, that a passage with a missing field is rejected, that every resolved office appears in the office list, that no passage exceeds the token limit, and that both indexes contain every passage.
Acceptance Condition	All verification checks pass.
Document and Integrate	The chunk store and both indexes. Chunking is not changed after this increment is accepted.

Once this increment is accepted, the chunking procedure is unchanged, as the gold labels of the evaluation set are tied to the passage identifiers. Only then will the evaluation query set (Table 11) be labeled. It is prepared by a different member of the research team than the one who drafts the training set, and is reviewed by the adviser and the validators and set aside unused until final evaluation.

Increment 3: QLoRA Behavioral Fine-Tuning

The third increment adapts how the model answers, not what it knows. The training set (Table 12) is drawn from the passages built in Increment 2. It teaches the model to answer only from the passages supplied, to attach the four-field citation, to refer or decline when the passages do not settle a question, and to keep the required response format.

Table 6

Increment 3 Development Plan

Process Step	Activity in This Increment
Define Requirements	A reviewed training set of 300 items with a held-out split set aside before training. No item overlaps the evaluation set, and abstention examples are drafted by a different member than the one who drafts the evaluation set. The adapter trains on the RTX 4060 without relying on paged optimizer state, and it loads at start-up on top of the 4-bit model.
Design	The training set composition; adapter configuration and training settings (rank, scaling factor, target layers, learning rate, number of epochs, random seed).
Build	Draft and review the training items, set aside the held-out split, and train the adapter with PEFT on the 4-bit model.
Verify	Run the held-out split through the adapted and the unadapted 4-bit model under identical prompts, passages, and decoding settings, and compare their outputs. Confirm that no training item appears in the evaluation set.
Acceptance Condition	Training completes within 8 GB; the adapter loads at start-up; on the held-out split the adapted model follows the response format and citation behavior at least as well as the unadapted model; no overlap is found.
Document and Integrate	One trained adapter, stored and versioned separately from the base weights, with its settings recorded.

The comparison in this increment is used only to decide whether the adapter is ready. The comparison reported as a result is Test 3.1, which repeats it on the evaluation set after the system is frozen.

Increment 4: Retrieval-Augmented Generation Pipeline

The fourth increment connects the knowledge base to the adapted model. A question is searched with BM25 and with the bge-small-en-v1.5 embedding model, the two ranked lists are combined by Reciprocal Rank Fusion, and the five highest-ranked passages are given to the model with their citation fields. A citation verifier then checks the answer's citation before it is shown.

Table 7

Increment 4 Development Plan

Process Step	Activity in This Increment
Define Requirements	Both retrievers return passages for a question; the two lists are combined into one ranking; the five highest-ranked passages and their fields reach the model. The verifier confirms that the cited document, office, year, and page belong to a passage the model was given, and an answer that fails is handled as an unsupported query. Dense retrieval runs on the processor.
Design	Tokenization and stemming for BM25; how many passages each retriever returns; the RRF constant; the layout of passages in the prompt; the citation format the verifier reads.
Build	Implement both retrievers over the Increment 2 indexes, the fusion step, prompt assembly, and the citation verifier.

Process Step	Activity in This Increment
Verify	On the development query set, compare BM25-only, dense-only, and RRF retrieval to set the retrieval settings. Confirm that a question containing a document control code retrieves the passage carrying it, that the passages given to the model come from the fused list, and that the verifier accepts a matching citation and rejects one naming a passage not supplied.
Acceptance Condition	All functional checks pass, and the retrieval settings are recorded.
Document and Integrate	A working cited-answer path: question → hybrid retrieval → adapted model → citation verifier.

Increment 5: Query Routing Mechanism

The fifth increment adds the two-gate router that decides each response path before the model is called. The evidence gate sends a question with sufficient retrieval evidence to Path A. Otherwise, the scope gate compares the question with the scope exemplar set (Table 13) and sends it to Path B for an office referral or to Path C for the fixed out-of-scope message.

Table 8

Increment 5 Development Plan

Process Step	Activity in This Increment
Define Requirements	Every question receives exactly one of the three paths. The scope gate runs only when evidence is insufficient. A Path B referral names only an office in the office list. Path C returns the fixed message without calling the model. The

Process Step	Activity in This Increment
	interface displays the response for the chosen path, and the route is logged.
Design	Normalization of the BM25 score; the evidence thresholds; the one-nearest-neighbor scope classifier and exemplar set; the referral and out-of-scope responses; the route display.
Build	Implement both gates, precompute the exemplar vectors, and add the route dispatcher and route logging.
Verify	Set the evidence thresholds using the development query set. Confirm that every development query receives exactly one path, that Path C never calls the model, that referrals name only listed offices, and that routes are logged.
Acceptance Condition	All verification checks pass, and the thresholds are recorded.
Document and Integrate	The complete prototype, ready for its settings to be frozen.

Knowledge Base and Datasets

The increments rely on one knowledge base and three sets of questions: the evaluation query set, the training set, and the scope exemplar set. Each is described below.

Knowledge Base Sources

The knowledge base is built only from the three official manuals the university issues to enrolled students (Table 9), so every answer can be traced to an institutional source.

Table 9
Knowledge Base Source Manuals

Source Manual	Edition	Coverage	Pages
Student Manual	2022	Admission and enrolment, academic guidelines, student services, code of conduct	120
Student Research and Innovation Manual	2023	Terminal program, final paper, journal publication, intellectual property, innovation	60
Internship Manual	2023	Internship management, required hours by program, internship forms	19
Total			**199**

Front matter with no student procedure, such as title pages, messages from officials, committee lists, signature forms, vision and mission statements, and section-divider pages, is removed before chunking.

Passage Record

Each passage is stored with the fields in Table 10. A passage missing any of the four citation fields is not added to the knowledge base.

Table 10

Passage Record Fields

Field	Purpose
Document title	Identifies the specific university manual or handbook from which the passage was extracted.
Issuing office	Specifies the administrative office or department responsible for the policy or section.
Edition year	Records the publication or revision year of the source manual to show which edition is cited.
Printed page	Indicates the exact page number in the physical/PDF manual for precise traceability.
Passage text	Contains the extracted, semantically cohesive chunk of text used for indexing and answer generation.

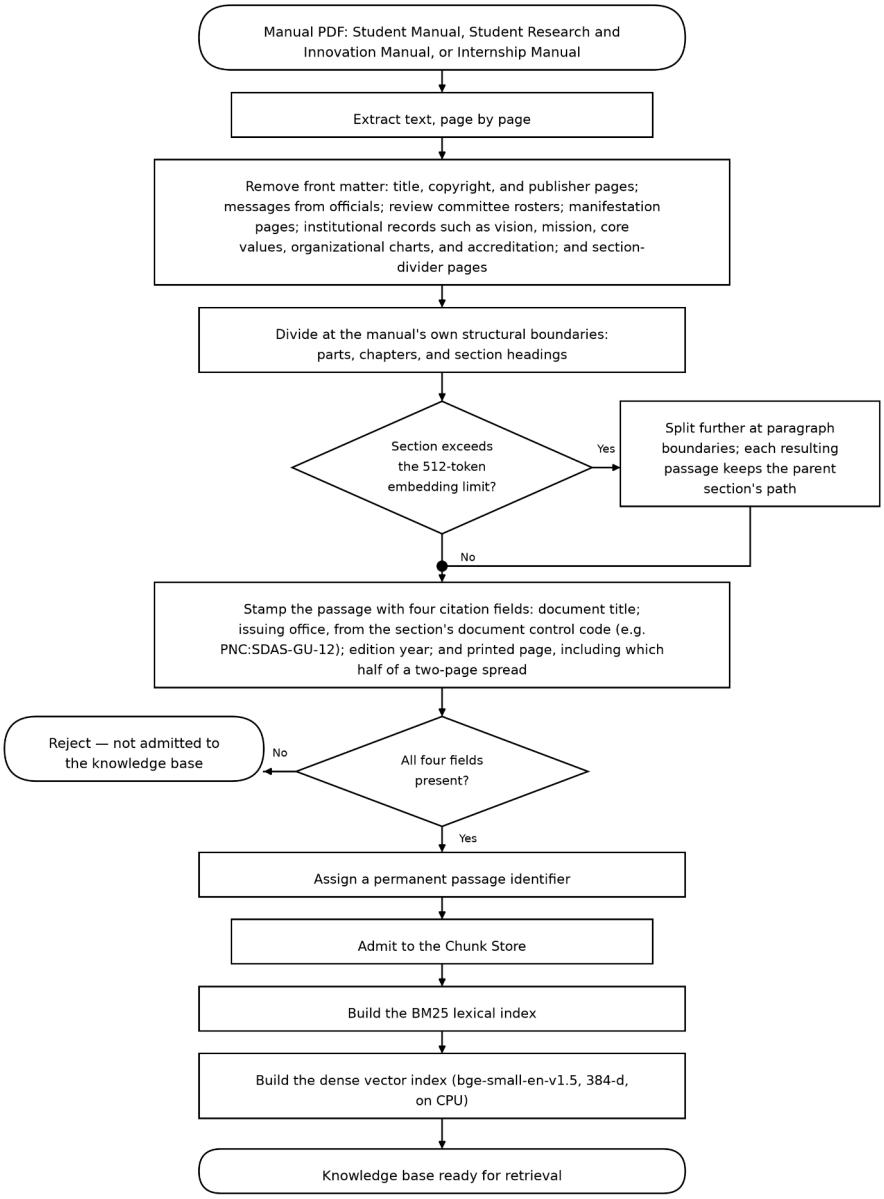


Figure 4. Knowledge Base Construction Procedure

Figure 4 shows how each manual is turned into stored passages, from text extraction to the two search indexes.

Evaluation Query Set

The evaluation query set has 80 questions in three strata, one for each response path (Table 11). The questions are fixed before evaluation and are not used during development.

Table 11
Evaluation Query Set Distribution

Stratum	Definition	Expected Response	Queries
Answerable	Answered by a passage in the knowledge base	Cited answer	40
In scope, no evidence	Within the manuals' subject matter but not answered by any passage	Office referral	25
Out of scope	Outside the manuals' subject matter	Out-of-scope message	15
Total			**80**

Training Set

The training set instructs the model on responding, rather than university material (Table 12). This consists of 300 training examples written with the help of a language model and then examined by a researcher against the source passage. A

held-out split is established prior to the training and none of the items is included in the evaluation query set.

Table 12

Training Set Composition

Example Type	What It Teaches
Grounded answer	Answer only from the supplied passages and cite them.
Referral	Say the manuals do not answer the question and name the responsible office.
Out of scope	Decline questions outside the manuals' subject matter.
Insufficient evidence	Withhold an answer when the passages do not settle the question.
Response format	Keep the required answer structure and citation placement.

Scope Exemplar Set

The scope exemplar set gives the router examples of in scope and out of scope questions (Table 13). It is used only when retrieval finds too little evidence, and it is kept separate from the evaluation query set.

Table 13

Composition of the Scope Exemplar Set

Category	Examples of Topics
In scope	Student policies and services, research and innovation, internship, and a student's own records
Out of scope	Other universities, general knowledge, entertainment, and personal advice

Operation and Testing Procedure

This section describes how the complete chatbot processes a student's query and how the team tests it using the test queries developed for this study. It covers how the system works and how results are recorded. The Evaluation Procedure contains the information pertaining to how the results will be validated to ensure correctness, groundedness, proper citation, and compliance with ISO/IEC 25010:2023.

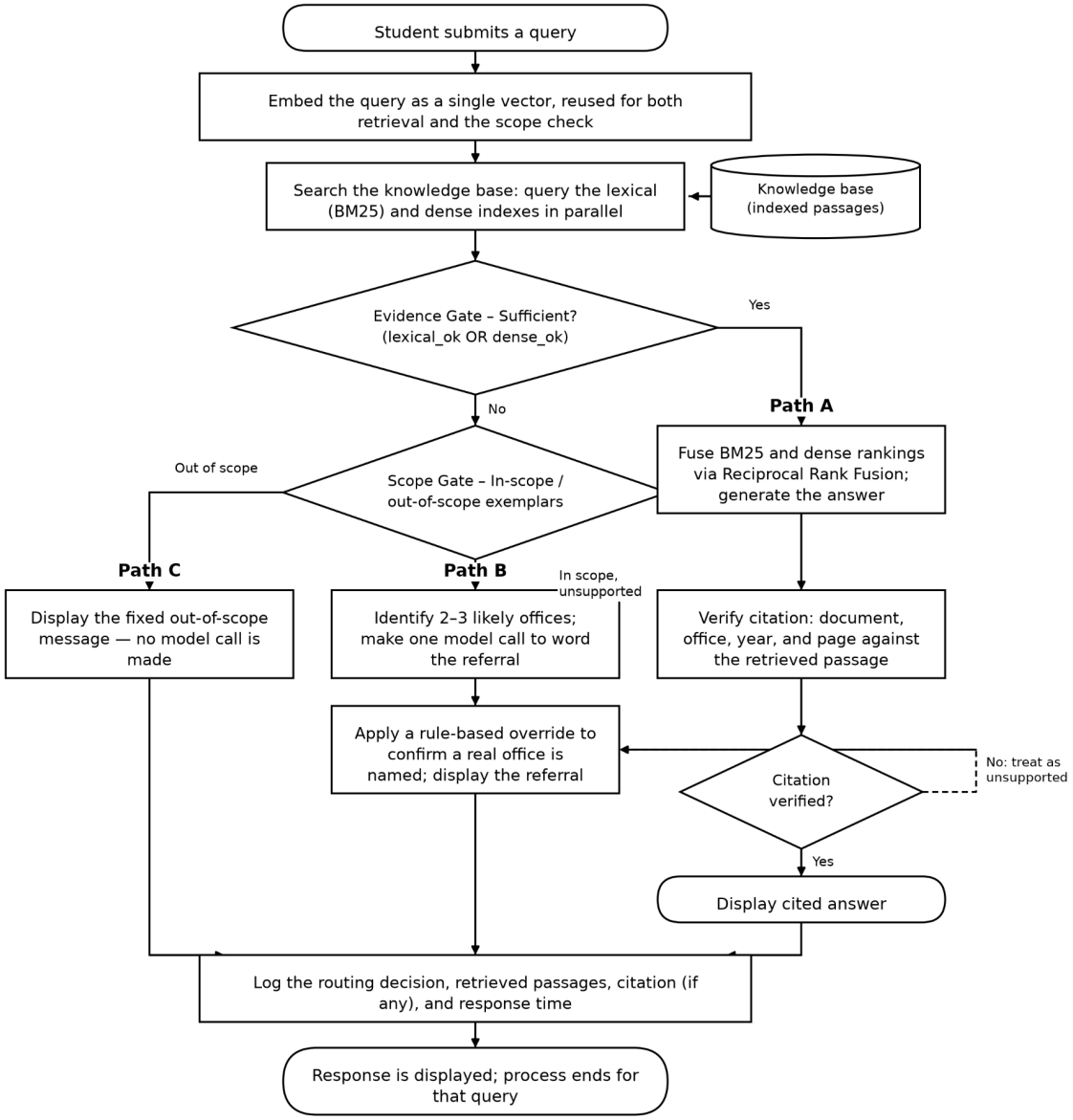


Figure 5: System Flowchart for Query Processing and Response Routing

Figure 5 shows the overall query-processing flow of the proposed system, which consists of nine steps:

1. **The student submits a query.** The process begins when a student types a question into the system.
2. **The system embeds the query.** A single vector representation of the question is created. This same vector is used later for both retrieval and the scope check, so the query is only embedded once per request.
3. **The Evidence Gate checks if there is enough evidence.** The system checks whether the lexical search (BM25) or the dense (meaning-based) search found a strong enough match — either one is enough to pass (lexical_ok OR dense_ok).
4. **If evidence is sufficient, Path A begins.** The lexical and dense results are combined using Reciprocal Rank Fusion, and the top-ranked passages are sent to the model, which generates one answer. Before the answer is shown, a citation checker confirms in code that the cited document, office, year, and page actually match a passage that was retrieved, and once verified, the cited answer is displayed to the student.
5. **If evidence is insufficient, the Scope Gate checks scope.** The system compares the query against a labeled set of in-scope and out-of-scope example questions.
6. **If the query is in scope but unsupported, Path B begins.** The system identifies two to three likely offices that could handle the concern, makes one short model call to word the referral, applies a rule-based override to ensure

the office name is a real one, and displays the referral message to the student.

7. **If the query is out of scope entirely, Path C begins.** A fixed, pre-written message is shown stating the question is outside what the system covers, and no model call is made for this path.

8. **The system logs the result.** Regardless of which path was taken, the routing decision, retrieved passages, citation (if any), and response time are recorded.

9. **The response is displayed**, and the process ends for that query.

Testing Procedure

Preliminary Calibration Run

Purpose. To characterize the response time and generation speed on the deployment machine before the formal testing begins, and to set the thresholds for Objective 1 against both this hardware characterization and an independently chosen ceiling.

1. Select ten calibration queries completely separate from the 80-query evaluation set.
2. Load the finished prototype to the deployment workstation with the network disabled
3. Submit each calibration query and log total response time, generation speed (tokens per second) and time-to-first-token.

4. Calculate the median response time and the min-max range for the ten runs. Report the median and range instead of a 95th percentile -- ten samples is too few to get a stable estimate of a percentile.
5. At the same time, before reviewing these results, set an a priori acceptability ceiling for response time, based on published helpdesk-chatbot latency norms or what a walk-up kiosk interaction can tolerate (e.g., a stated ceiling such as ≤ 60 seconds).
6. The two references that will be used to evaluate Objective 1 are the hardware calibrated median/range and the a priori ceiling. Both figures should be recorded in Chapter III before any evaluation query is run.

Objective 1 — Offline Deployment of the 4-Bit Quantized Model

Test 1.1 — Offline Operation

Purpose. To determine if the prototype is able to work without being connected to the internet or any external service.

1. Disable the Wi-Fi and wired network access on the host computer and make sure there is no external connection.
2. Start the local server and load the model, adapter, BM25 index, and embedding files from local storage.
3. Start the local server and load the model, adapter, bm25 index, and embedding files locally.
4. Monitor outbound network activity during execution and log any external connection attempts.

5. For each query, indicate whether the system provided a complete answer from local components only.

Test 1.2 — Response Time

Purpose. We report the total time to generate and display a response, separately for each routing outcome so that near-instant out-of-scope replies are not pooled with full generations.

1. Load model, adapter and retrieval index into memory before timing starts (warm state).
2. For each query, record the clock when the query is submitted, and again when the full answer (and citation, if any) is done displaying.
3. Group the results by routing outcome—Path A (cited answer), Path B (office referral), Path C (out-of-scope)—and calculate the median response time for each group. For Path A we report only a 95th percentile, where the group is large enough (≈40 queries) for the estimate to be meaningful; for Paths B and C we report the median and full range.
4. At the first startup make one separate cold-load measurement and report it separately from the steady-state figures.
5. A “timing-sensitive query” is any pre-defined Path A query; each is submitted three times and the average of the three is used, so as to reduce one-off variation. The response time for Path B and C queries is only the first measurement as their response is independent of generation.

Test 1.3 — Generation Speed

Purpose. To measure the speed it takes for the model to generate its response, separate from retrieval and routing.

1. For each Path A query, record the number of output tokens, generation time only (excluding retrieval and routing), and time-to-first-token.
2. Compute the number of tokens produced per second for each query, and take the median over the set of queries.

Test 1.4 — Peak Memory Use

Purpose. Check whether the quantized model and adapter will fit into the 8 GB video memory limit.

1. Begin monitoring VRAM via the graphics card's allocation counters before the first query is submitted.
2. Record peak VRAM allocation during answer generation and system memory over the same time period.
3. Do this for the entire query set and take the maximum seen.
4. Check that the model files, indexes and manuals are stored on the local drive and log the model load time at startup.

Objective 2 — Knowledge Base Construction

Test 2.1 — Citation Metadata Completeness and Accuracy

Purpose. To determine both (a) whether every stored passage carries a complete citation record and (b) whether that record is actually correct — completeness alone does not establish that the metadata is right.

1. Run automated assertions over every passage after the build completes: check that each carries all four citation fields (title, issuing office, year, page/section), that no passage exceeds the chunk-size cap, and that a test passage missing a field is rejected by the build.

2. Compare document and page counts in the knowledge base against the source manuals.
3. Draw a random sample of 50 stored passages, stratified across source documents.
4. For each sampled passage, manually check its citation fields against the actual source PDF: does the cited page really carry that text, and is the cited office really the issuing office for that section?
5. Record the number of sampled passages whose citation record is fully correct.

Test 2.2 — Issuing Office Resolution

Purpose. To determine whether the issuing office recorded for each passage is both a real office and the correct one for that section.

1. Obtain the confirmed organizational chart as the reference list of valid offices.
2. Run the versioned section-to-office lookup table over every stored passage and check that each resolved office appears on the chart.
3. From the same 50-passage sample drawn in Test 2.1, manually check whether the resolved office is the correct issuing office for that section (not merely a valid office name).

Objective 3 — QLoRA Behavioral Fine-Tuning

Test 3.1 — Baseline Comparison (Pre/Post Fine-Tuning)

Purpose. To establish that QLoRA fine-tuning, rather than the base model or the retrieval pipeline, produced the behavioral change being claimed. Without this run, none of the Objective 3 numbers below can be attributed to fine-tuning.

1. Before applying the QLoRA adapter, run the base 4-bit quantized model (adapter disabled) on the same 40 answerable queries and the same 25 no-evidence queries used in Tests 3.2–3.4.
2. Score the base model's outputs using the same procedures as Tests 3.2 to 3.4: strict/lenient correctness, instruction-level and prompt-level compliance, decline rate, and fabricated-citation rate.
3. Repeat the same scoring on the fine-tuned model's outputs (this is simply Tests 3.2–3.4, reused here for comparison).
4. Report a single pre/post table with both sets of figures side by side.

Test 3.2 — Answer Correctness

Purpose. To determine whether the system's answers state the same facts as the pre-registered reference answers.

Prepare a reference answer for each of the 40 answerable queries from its gold passage, before testing begins.

1. Submit each query to the completed system and capture the generated answer.
2. Score each answer against its reference answer and supporting passages using an LLM judge applying the study's rubric, under both a strict and a lenient reading.
3. Have a human rater independently score a fixed sample of 15 of the 40 answers (drawn to include cases across both readings).
4. Measure agreement between the LLM judge and the human rater using Cohen's kappa (κ).

Test 3.3 — Instruction Following

Purpose. To determine whether responses obey the required output format, independent of factual correctness.

1. Define the format checklist in advance: direct answer present, required structure followed, citation carrying document title, issuing office, year, and page.
2. Run the automated constraint-checker script over every generated response.
3. Score each response at the instruction level (each rule satisfied) and at the prompt level under a strict reading (every rule satisfied).
4. Record format failures separately from factual errors.

Test 3.4 — Handling of Unsupported Queries

Purpose. To determine whether the fine-tuned model itself declines to answer when the passages it receives do not support one, with the router's gates disabled but retrieval still running.

1. Take the 25 in-scope, no-evidence queries.
2. Submit each query with the gates disabled: retrieval still runs and the top-k passages are supplied to the model exactly as in normal operation, but the Evidence Gate and Scope Gate do not intervene in the routing decision.

Test 3.5 — Filipino / Taglish Input Behavior (Exploratory, Not Scored)

Purpose. To document what the system does when a student writes in Filipino or Taglish, since the deployment site (University of Cabuyao) will receive such input even though the system's designed scope is English.

1. Submit a small fixed set of Filipino/Taglish queries (e.g., “Paano mag-enroll?”, “Ano po ang requirements sa TOR?”) that mirror answerable, no-evidence, and out-of-scope categories.
2. Record the routing outcome, the language of the response, and whether the response is coherent, regardless of correctness.
3. Report this as a documented limitation with example transcripts, not as a pass/fail measure — the system’s scope statement explicitly limits it to English, so this test does not carry an acceptance criterion.

Objective 4 — Retrieval-Augmented Generation Pipeline

Test 4.1 — Retrieval Ablation (BM25-only, Dense-only, RRF)

Purpose. To show that the hybrid fusion step contributes something the individual retrievers do not, before crediting RRF for the results in Test 4.2.

1. For each of the 40 answerable queries, run retrieval three ways: BM25 alone, dense retrieval alone, and the full RRF fusion.
2. Compute Recall@5 and MRR for each of the three conditions using the same gold passages and the same depth used in Test 4.2.
3. Report a three-row table (BM25 / Dense / RRF) on Recall@5 and MRR.

Test 4.2 — Retrieval Accuracy

Purpose. To determine whether the retrieval pipeline returns the passages that actually answer the question.

1. Assign one or more validated gold passages to every answerable query before testing.

2. Run the retriever for each query through BM25, dense retrieval, and RRF, and capture the fused ranked list to a depth of 10.
3. Check whether at least one gold passage appears in the top 5 fused results, and record the rank of the first relevant passage.
4. Identify, in advance, at least 3 of the 40 answerable queries that contain a document control code, and for each confirm that the passage carrying that code is retrieved in the top 5.

Test 4.3 — Answer Groundedness

Purpose. To determine whether the system's answers stay within the evidence actually supplied to the model.

1. Capture the passages supplied to the model for each answerable query alongside the generated answer.
2. A designated member of the research team (the same rater used in Test 3.2, for consistency) segments each generated answer into its individual factual claims, following a fixed segmentation rule: one claim per independently verifiable statement, split at clause boundaries where the statement could be true or false on its own.
3. Verify each claim directly against the supplied passages; mark an answer as grounded only when every claim it makes is supported.
4. Record any claim that is true in general but absent from the supplied passages as ungrounded.

Test 4.4 — Citation Accuracy

Purpose. To determine whether the citation attached to an answer points to the passage that actually contains the stated information.

1. Capture the citation attached to every cited answer.
2. Validate the cited page/section, title, year, and office against the passage record in the knowledge base, and confirm the cited passage was among those supplied to the model for that query.
3. Supply the verifier with one citation naming a passage not given to the model, and confirm it rejects the citation and routes the answer as unsupported.
4. Count a citation as inaccurate when it is correctly formatted but points to the wrong document, page, office, or year.

Objective 5 — Query Routing Mechanism

Test 5.1 — Routing Accuracy

Purpose. To determine whether each question is directed to the response path it should take.

1. Assign an expected route (cited answer, office referral, or out-of-scope) to every one of the 80 queries before testing.
2. Submit each query through the complete system with both gates active and record the path actually taken.
3. Confirm every query produces exactly one of the three outcomes, and that out-of-scope questions return the fixed message without reaching the model.
4. Log the number of generator calls per query as a cross-check on the routing record.
5. Build a 3×3 confusion matrix (expected route × actual route) so the direction of every error is visible.

Test 5.2 — Dangerous Error Rate and Referral Correctness

Purpose. To determine (a) how often an out-of-scope question is wrongly treated as answerable, the most consequential routing error, and (b) whether referrals name the right office, not merely a valid one.

1. Isolate the 15 out-of-scope queries from the Test 5.1 confusion matrix.
2. Count how many were routed to a cited answer instead of being declined, and record each case with the gate outcome that allowed it through.
3. For the 25 office-referral queries, compare the office named in each referral against a pre-assigned expected office (not merely a check against the valid-office list).

Objective 6 — ISO/IEC 25010:2023 Quality Characteristics

These tests do not generate new measurements; they organize the results already obtained under Objectives 1–5. The expert questionnaire that separately evaluates functional suitability, performance efficiency, and reliability is administered under the Evaluation Procedure, not here — this section reports only the objective, log-derived side of Objective 6.

Test 6.1 — Functional Suitability (Objective Measures)

1. Collect answer correctness, instruction following, and unsupported-query handling from Objective 3.
2. Collect retrieval accuracy, groundedness, and citation accuracy from Objective 4.
3. Collect routing accuracy and dangerous error results from Objective 5.
4. Map each measure to its registered threshold and record which are met.

Test 6.2 — Performance Efficiency

1. Collect response time, generation speed, and peak memory results from Objective 1.
2. Capture a latency breakdown per query — time spent in routing, retrieval, and generation, in milliseconds and as a percentage of total response time.
3. Confirm peak VRAM stays within the card's capacity with the interface running.
4. Compare each figure against the calibrated and a priori thresholds.

Test 6.3 — Execution Stability Check

Purpose. Consistent with the Evaluation Procedure, this run is a testing sanity check, not a separately scored reliability measure — reliability itself is reported through Test 3.4 (unsupported-query handling) and Test 5.2 (dangerous error rate).

1. Run the full 80-query set continuously in one session, with query order randomized rather than fixed, so thermal drift is not confounded with query difficulty.
2. Record any crash, freeze, timeout, or failure to return a response.
3. Monitor temperature and check whether response times (by position in the run) degrade as the run proceeds.
4. Confirm every run logs route, retrieved passages, citation decision, response time, generation speed, and peak memory.
5. Set aside any run with an unexpected error for manual review rather than discarding it.

6. Repeat a subset of 15 queries under the fixed seed and temperature (T = 0) and compute the exact-match rate between repeated runs, rather than requiring bitwise-identical text — GPU kernel behavior does not guarantee bitwise determinism even at T = 0. For non-exact matches, separately check whether the route and cited passage stayed the same.

Scope of This Procedure

This procedure is complete once every test above has been run and logged as described in Test Execution Records. It defines how each component is exercised and how measurements are captured; the interpretation of those measurements against the study's acceptance standards, and their organization under ISO/IEC 25010:2023, is presented in the Evaluation Procedure that follows.

Note: The response-time and generation-speed thresholds referenced throughout this section are fixed by the Preliminary Calibration Run against both a hardware-calibrated figure and an independent a priori ceiling. The remaining thresholds are taken from Tables 16 to 21. Offline Operation Rate, the metadata and office-resolution build checks, Completion Rate, and Exact-Match Rate are execution checks internal to this procedure rather than registered evaluation criteria, and are reported as build-integrity results rather than measured against a pre-registered target.

Evaluation Procedure

The developed offline university helpdesk chatbot will undergo a process of assessment in the field of technical performance and system evaluation. Moreover, technical performance is defined as the ability of the chatbot to respond correctly to queries related to the university and provide references to the answers it provides, check the questions that cannot be answered by the chatbot, and send them to the proper university departments. System evaluation reflects the effectiveness of the chatbot in terms of its functional appropriateness, effectiveness, and reliability, which is assessed by a certain group of experts in this area.

General Evaluation

The evaluation uses the completed records from the 80-question dataset: 40 answerable questions, 25 in-scope questions without sufficient evidence, and 15 out-of-scope questions. Each result is reported using the appropriate denominator and includes the numerator, percentage, and, for proportion measures, a Wilson 95% confidence interval.

For a proportion $\hat{p} = \frac{x}{n}$, the Wilson 95% confidence interval is calculated as:

$$Wilson\ Ci = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

where $\hat{p} = \frac{x}{n}$ and z = 1.96 for the 95% confidence level. Results will be reported as counts, percentages, and applicable summary statistics (median, range, 95th percentile, mean, or standard deviation).

Objective 1 — Offline Deployment of the 4-Bit Quantized Model

Data from Tests 1.1 through 1.4 are examined to confirm if the implemented model functions in offline mode and meets the specified performance thresholds of the Preliminary Calibration Run.

Offline Operation

The Offline Operation Rate is calculated as:

$$OOR = \frac{S}{T} X\ 100$$

where S is locally completed queries and T is the total number of tested queries. Pass: OOR = 100%, all files load locally, and no external connection is recorded.

Response Time

For each query, response time is calculated as:

$$RT = t_2 - t_1$$

where t_1 is query submission and t_2 is a complete response display. Path A passes when its median and 95th percentile meet the calibrated limits; Paths B and C use median and range. Cold-start time is separate.

Generation speed and memory

Generation speed is calculated as:

$$GS_i = \frac{N_i}{G_i}$$

where N_i is the number of generated tokens and G_i is generation time in

seconds, excluding retrieval and routing. The median is compared with the calibrated thresholds.

Peak video memory is calculated as:

$$VRAM_{peak} = max\,(VRAM_i),\ i = 1, \ldots, 80$$

The result passes when peak VRAM is below 8 GB, with a target of no more than 7.5 GB, and no memory crash or out-of-memory failure occurs. System memory, generation speed, and response-time summaries are reported alongside the pass/fail decision.

Objective 2 — Knowledge Base Construction

The records from Tests 2.1 and 2.2 are used to evaluate the integrity and accuracy of the knowledge-base metadata.

Metadata completeness and citation verification

Metadata completeness is calculated as:

$$MC = \frac{P_{ok}}{P_{total}} \times 100$$

where P_{ok} is the number of stored passages containing all required fields (document title, issuing office, year, and page or section) and P_{total} is the total number of stored passages. The build passes the completeness check when MC = 100% and no passage exceeds the chunk-size limit. Citation verification accuracy is calculated from the 50-passage validation sample as:

$$CV = \frac{S_{correct}}{S_{checked}} \times 100$$

where $S_{correct}$ is the number of sampled passages whose citation metadata matches the source document and $S_{checked}$ = 50. The result passes when CV ≥ 95%. The Wilson 95% confidence interval will be reported because the sample is limited.

Issuing-office resolution

Office Resolution Rate is treated as a build-integrity check:

$$ORR = \frac{P_{valid}}{P_{total}} \times 100$$

where P_{valid} is the number of passages resolving to an office listed on the verified organizational chart. The expected result is ORR = 100%.

Office Resolution Accuracy is calculated from the manually checked sample:

$$ORA = \frac{S_{correct}}{S_{checked}} \times 100$$

The result passes when ORA ≥ 95%, with its Wilson 95% confidence interval reported. A valid office name that is incorrect for the particular section is counted as an error.

Objective 3 — QLoRA Behavioral Fine-Tuning

The data from Tests 3.1 through 3.4 will show if the targeted behaviors change when the QLoRA adapter is used, where Condition A refers to the deployed model with the adapter present and Condition B refers to the same quantified base model without the use of the adapter. To assess the results, the obtained responses will be evaluated under the blinded procedure outlined in the Testing Procedure.

Answer correctness

For the 40 answerable questions, answer correctness is calculated as:

$$AC = \frac{C}{Q} \times 100$$

where C refers to the total number of answers marked as correct, while "Q" is equal to 40. The two types of scores are defined as follows: strict correctness means that the information presented contained no material mistakes, contradictions, and omissions and lenient correctness means that some omissions in the text are allowed since a respondent still provided the correct answer to the question. The acceptable cutoff levels are strict AC ≥ 80% and lenient AC ≥ 90%, both with the Wilson CI at the confidence level of 95%.

Instruction following

Instruction-Level Compliance and Strict Prompt-Level Compliance are calculated as:

$$ILC = \frac{R_{ok}}{R_{total}} \times 100$$

$$SPC = \frac{P_{ok}}{P_{total}} \times 100$$

ILC measures satisfied individual rules; SPC measures fully compliant responses. Thresholds: ILC ≥ 95% and SPC ≥ 85%. A factually correct response in the wrong format remains an instruction-following failure.

Unsupported-query handling

For the 25 in-scope, no-evidence questions, the Decline Rate is:

$$DR = \frac{D}{Q} \times 100$$

where D is the number of queries correctly declined and Q = 25. The result passes when DR ≥ 90%. The Fabricated-Citation Rate is:

$$FCR = \frac{F}{Q} \times 100$$

where F is the number of unsupported or fabricated citations among the 25 queries. Zero observed fabricated citations will be reported as FCR = 0% with a Wilson interval and interpreted as “no fabricated citations observed,” not as proof that the true population rate is zero.

QLoRA comparison and rater agreement

For every metric, the observed change is:

$$\Delta M = M_A - M_B$$

where M_A is the result with the adapter and M_B is the result without it. A positive difference indicates improvement when higher values are better; for FCR and other error rates, a negative difference indicates improvement. The report includes both condition values, the difference, and the number of queries that improved, stayed the same, or worsened. No causal or statistical-significance claim is made.

Agreement between the LLM judge and the human rater is calculated using Cohen’s kappa: $\kappa = \frac{P_0 - P_e}{1 - P_e}$

where Po is observed agreement and Pe is chance-expected agreement. A value of κ ≥ 0.61 is interpreted as substantial agreement. If the threshold is not reached, disagreement cases are reviewed, the rubric is clarified, and the affected results are re-coded and documented.

Filipino or Taglish behavior is exploratory only. It is reported descriptively with sample observations and is not included in the pass/fail computation because the system scope is English.

Objective 4 — Retrieval-Augmented Generation Pipeline

The records from Tests 4.1–4.2 are evaluated independently of the generated answer. Retrieval is scored using the verified gold passages and the recorded rankings.

Retrieval ablation and retrieval accuracy

Recall@5 is calculated as:

$$Recall@5 = \frac{H}{Q} \times 100$$

where H is the number of answerable queries with at least one gold passage in the top five and Q = 40. Mean Reciprocal Rank is:

$$MRR = \frac{1}{Q} \sum_{i=1}^{Q} \frac{1}{rank_1}$$

where $rank_1$ is the rank of the first relevant passage; a query with no relevant passage in the top five contributes zero. BM25-only, dense-only, and RRF results are displayed in the same table using the same queries and labels. RRF is described as beneficial only when its observed Recall@5 or MRR exceeds the

relevant single-retriever baseline.

The retrieval acceptance thresholds are Recall@5 ≥ 85% and MRR ≥ 0.70. Recall@5 includes a Wilson 95% confidence interval. Control-Code Retrieval Rate is also reported as:

$$CCR = \frac{C_{hit}}{C_{total}} \times 100$$

and must equal 100% for the pre-identified control-code queries.

Groundedness

Answer-Level Groundedness is calculated as:

$$AG = \frac{A_{ok}}{A_{total}} \times 100$$

where A_{ok} is the number of answers whose factual claims are all supported by the supplied passages. Claim-Level Groundedness is:

$$CG = \frac{K_{ok}}{K_{total}} \times 100$$

where K_{ok} is the number of supported claims and K_{total} is the total number of claims. The thresholds are AG ≥ 85% and CG ≥ 95%, each with a Wilson 95% confidence interval. A fact that is generally true but absent from the retrieved evidence is not counted as grounded.

Citation accuracy

Citation Accuracy is calculated as:

$$CA = \frac{C_{accurate}}{C_{checked}} \times 100$$

A citation is accurate only when its title, issuing office, year, and page or section correspond to a supplied supporting passage. A correctly formatted citation pointing to the wrong source remains inaccurate. The result passes when CA ≥ 95%, with a Wilson 95% confidence interval.

Objective 5 — Query Routing Mechanism

The records from Tests 5.1 and 5.2 are evaluated by comparing the expected route with the actual route. The three routes are Path A: cited answer, Path B: office referral, and Path C: out-of-scope response.

Routing accuracy

Routing Accuracy is calculated as:

$$RA = \frac{N_{correct_routes}}{N_{all_queries}} \times 100$$

The three-by-three confusion matrix shows expected routes as rows and predicted routes as columns. Diagonal cells represent correct classifications, while off-diagonal cells identify misrouting patterns. Macro-F1 and the F1 score for each path are reported from the same matrix:

$$Macro - F1 = \frac{F1_A + F1_B + F1_C}{3}$$

The thresholds are RA ≥ 85% and Macro-F1 ≥ 0.80, with a Wilson interval reported for RA. Out-of-scope queries must not reach the model and each query must produce one route only.

Dangerous errors and referral correctness

A dangerous error is an out-of-scope query incorrectly treated as answerable, or another routing error that may direct a user to an unsupported answer. The Dangerous Error Rate is:

$$DER = \frac{E_{dangerous}}{Q_{out}} \times 100$$

where $E_{dangerous}$ is the number of dangerous errors and Q_{out} = 15 is the number of out-of-scope queries. Because one error already exceeds the intended 5% level for this stratum, the binding criterion is zero observed dangerous errors ($E_{dangerous}$ = 0). All such cases are listed individually.

Referral Correctness is:

$$RC = \frac{O_{correct}}{O_{expected}} \times 100$$

where $O_{correct}$ is the number of referrals naming the pre-assigned correct office and $O_{expected}$ is the number of expected office referrals. The result passes when RC ≥ 90%. A valid but wrong office is counted as an incorrect referral, although it is not automatically a dangerous error.

Objective 6 — ISO/IEC 25010:2023 Quality Characteristics

Objective 6 organizes both the objective results from Objectives 1–5 and the separate expert questionnaire. The two evidence types are not combined into an unsupported single score.

Table 14

ISO/IEC 25010:2023 Characteristics and Evidence Used

ISO/IEC 25010:2023 Characteristic	Objective Evidence Used	Expert Questionnaire Focus
Functional suitability	Answer correctness, instruction compliance, decline rate, retrieval, groundedness, citation accuracy, routing, and referral correctness	Whether intended helpdesk functions are provided correctly
Performance efficiency	Response time, generation speed, time-to first-token, VRAM, and system memory	Whether responses are timely and resources are used efficiently
Reliability	Offline operation, complete logs, unsupported-query handling, dangerous errors, and system-error records	Whether the system behaves consistently and handles failures appropriately

Expert evaluation scoring

Three to five experts evaluate the frozen system: 2 technical experts and one to three domain experts from offices represented in the knowledge base. Their ratings are descriptive and are not generalized to all students or university users. The questionnaire uses a five-point Likert scale: 5 — Excellent, 4 — Very Good, 3 — Good, 2 — Fair, and 1 — Poor.

The weighted mean is:

$$WM = \frac{\Sigma(f \times x)}{N}$$

where f is the frequency of each rating, x is the numerical rating, and N is the total number of responses. Means are computed separately for functional suitability, performance efficiency, reliability, and the overall questionnaire.

Table 15

Weighted Mean Interpretation Scale

Weighted Mean	Interpretation
4.21 – 5.00	Excellent
3.41 – 4.20	Very Good
2.61 – 3.40	Good
1.81 – 2.60	Fair
1.00 – 1.80	Poor

The expert results are reported as descriptive system-quality evidence. They do not replace the objective acceptance thresholds for correctness, retrieval, routing, response time, memory, or error rates.

Reporting and Interpretation of Results

The final report presents: (1) the dataset and validation records; (2) Objective 1 performance results; (3) Objective 2 knowledge-base results; (4) Objective 3 correctness, compliance, unsupported-query, QLoRA, and agreement results; (5) Objective 4 retrieval, groundedness, and citation results; (6) Objective 5 routing, referral, and dangerous-error results; and (7) Objective 6 ISO/IEC 25010:2023 questionnaire results.

Each metric is labeled Pass, Fail, Descriptive, Exploratory, or Limitation. A metric is marked Pass only when its recorded point estimate meets the pre-established threshold and the required scoring procedure is valid. Confidence intervals show uncertainty but are not used to change the pre-established pass/fail rule after results are observed. Results from different denominators are not averaged together. The discussion distinguishes observed prototype performance from general claims about the wider university population.

Ethical Measures

Expert participation is voluntary and based on informed consent. Only professional roles and years of experience are collected, and these details are aggregated. Evaluators are identified in the manuscript as Expert 1, Expert 2, and so forth. Ratings are securely stored, used only for the system evaluation, and not linked to individuals in published outputs. No student information is collected, stored, or processed.

The evaluation will cover answer quality, response routing, technical performance, reliability, software quality, and evaluator feedback.

1. Evaluate the test dataset by preparing 80 questions from official University of Cabuyao documents. The questions will include answerable questions, in-scope-without-evidence questions, and out-of-scope questions. Each question will be accompanied with reference answer, expected path, and supporting documents which will be reviewed by the adviser and qualified validators.
2. Evaluate the testing environment through the loading of quantized Llama 3.2 3B model, QLoRA adapter, hybrid retrieval modules, knowledge base and path router on the designated computer. Access to the internet or external cloud-based services will be disabled during testing.
3. Identify the quality of answers provided by the chatbot by comparing the generated responses with those from reference answers and supporting documents. The evaluation will consider answer correctness, groundedness, citation accuracy, and compliance with the required response format.
4. Evaluate the routing efficiency through comparing the predicted route with the expected route: Path A (Cited Answer), Path B (Office Referral), or Path C (Out-of-Scope Response). Routing accuracy and a confusion matrix will be used to identify correct classifications and misrouting patterns.
5. Determine the actual performance of the system through capturing the response time, generation speed, peak GPU memory, peak system memory, retrieved evidence, citation and system errors. Response time will be analyzed using its median and 95th percentile; generation speed and peak memory will be analyzed using their mean and standard deviation; and pass/fail measures will be analyzed as frequency and percentage.

6. Check execution stability by repeating selected questions under identical conditions and recording any incomplete responses, retrieval failures, or system errors encountered. These repeated runs are a testing sanity check rather than a reported measure — reliability itself is reported through handling of unsupported queries (Objective 3) and the dangerous-error rate read from the routing confusion matrix (Objective 5).
7. Gather evaluator feedback by allowing selected evaluators to use the chatbot and complete a researcher-developed questionnaire based on ISO/IEC 25010:2023. The questionnaire will assess functional suitability, performance efficiency, and reliability using a five-point Likert scale, where 5 means Excellent and 1 means Poor. The questionnaire results will be appropriate descriptive statistics.
8. Assess the overall suitability of the chatbot by analyzing the technical results and evaluator feedback. The findings will identify the system's strengths, limitations, and areas for improvement in its intended university helpdesk application.

Technical Performance Evaluation

Technical performance will be determined from the results of the 80 question test set. Each response will be checked against its reference answer and official supporting document. Answer correctness will determine whether the information is accurate, groundedness will determine whether factual claims are supported by retrieved evidence, and for the citation accuracy, it will determine whether the cited document and page correctly correspond to the source. The response path taken by the system will also be compared with the expected path.

The researchers will record response time, generation speed, peak GPU memory, peak system memory, and errors for each query. Selected questions will be repeated to check execution stability. The recorded results will be analyzed using the median and 95th percentile for response time, the mean and standard deviation for generation speed and memory usage, routing accuracy, and a confusion matrix.

System Evaluation

System quality will be evaluated using a researcher-developed questionnaire based on selected ISO/IEC 25010:2023 characteristics: functional suitability, performance efficiency, and reliability. In order to fill the questionnaire, evaluators should first observe how the chatbot responds to queries and perform, including its answering, citation, and routing functions, before completing the questionnaire.

The evaluation will involve three to five experts. Among them, two should be technical experts (IT or Computer Engineering faculty, or university IT staff) and one to three domain experts (staff from the offices whose manuals the system draws from, such as the registrar, student affairs, or admissions). No students will participate in the evaluation process, consistent with the study's decision not to make any usability claim about the web interface. Once the tests have been completed under the Testing Procedure and if the system is still frozen, the expert will undergo a 30-minute guided demonstration using a fixed script of 15 questions drawn from a total of 80 questions (8 answerable, 4 no evidence, 3 out of scope).

The questionnaire will evaluate the following characteristics:

1. Functional suitability — the extent to which the chatbot provides the intended helpdesk functions, including answering questions, retrieving supporting information, displaying citations, and selecting the appropriate response path;

2. Performance efficiency — the extent to which the chatbot provides responses within an acceptable time while using available hardware resources efficiently;

3. Reliability — the extent to which the chatbot performs consistently, produces complete responses, avoids system errors, and handles unsupported questions appropriately.

The evaluators' responses will be rated using a five-point Likert scale: 5—Excellent, 4—Very Good, 3—Good, 2—Fair, and 1—Poor. The weighted mean for each characteristic and for the overall evaluation will be computed using the following formula:

$$Weighted\ Mean\ = \frac{\sum(f \times x)}{N}$$

Where:

- f represents the frequency of each rating
- x represents the numerical value of the rating
- N represents the total number of evaluator responses.

The calculated weighted means will be interpreted using a five-point scale. The analysis of technical performance results and system evaluation results will be treated separately in order to avoid confusion of the objective system measurements and evaluator's view.

The overall effectiveness of the chatbot will be determined through analysis of technical performance results and the evaluator feedback. The results will show both the strengths, limitations, reliability, and areas for improvement of the developed offline university helpdesk chatbot for its intended use at the University of Cabuyao.

Ethical Measures

Participation in the assessment will be on a voluntary basis and with informed consent. Personal information will not be collected beyond the expert's professional role and years of experience, which will be aggregated. All experts will be named only as Expert 1 to Expert N in the paper. Ratings will be securely stored and used exclusively for Objective 6, and will not be linked to any individual in any published output. There will be no data collection, storage or processing of student's information at any point in the study.

Evaluation Criteria

The proposed university helpdesk relies on a localized language model 4 bits of quantization along with QLoRA and Retrieval-Augmented Generation (RAG). As the system is still under development, its performance tests are not connected to any empirical data. The tables below outline the metrics, testing procedures, and target benchmarks designed to guide and validate the project throughout its development phases.

Table 16

Evaluation Matrix for Objective 1: Quantized Offline Model Deployment and System Efficiency Metrics

Evaluation Criterion	Metric	Proposed Testing Method	Target Benchmark
Peak VRAM	Peak memory	Track GPU VRAM	Must remain under **8**

Consumption	allocation (GB)	software allocation counters during response generation	**GB VRAM**, target peak $\leq$ $7.5GB$
Response Latency	Time-to-complete response (seconds), median 95th percentile	Measure elapsed time from prompt submission to complete text output	Median and 95th-percentile thresholds to be fixed from the 10-query calibration run in the Testing Procedure, recorded before evaluation begins
Generation Speed	Output throughput (tokens/second), time-to-first-token	Calculate generated text tokens produced per second	Threshold to be fixed from the same calibration run described below

Table 17

Evaluation Matrix for Objective 2: Knowledge Base Construction and Citation Metadata Integrity

Evaluation Criterion	Metric	Proposed Testing Method	Target Benchmark
Citation Metadata Completeness	Metadata Completeness (MC) — percentage of stored passages carrying all four citation fields	Automated assertions run over every passage after the build completes	MC = 100% (reported as a build-integrity check)
Citation Metadata Accuracy	Citation Verification accuracy (CV) — percentage of sampled passages	Manual check of a random 50-passage sample, stratified across source	CV ≥ 95%, with a Wilson 95% CI

	whose citation record matches the source document	documents, against the source PDFs	
Issuing Office Resolution	Office Resolution Rate (ORR) and Office Resolution Accuracy (ORA)	Versioned section-to-office lookup run over every passage and checked against the confirmed organizational chart; correctness verified on the same 50-passage sample	ORR = 100% (build-integrity check); ORA ≥ 95%, with a Wilson 95% CI
Corpus Integrity	Document and page counts; passage size	Compare knowledge base counts against the source manuals; verify no passage exceeds the chunk-size cap	Counts match the source manuals; no passage exceeds the 512-token cap

Table 18

Evaluation Matrix for Objective 3: QLoRA Behavioral Fine-Tuning, Instruction Compliance and Unsupported Query Handling

Evaluation Criterion	Metric	Proposed Testing Method	Target Benchmark
Answer Correctness	LLM-judge-rated correctness against reference answers (strict/lenient), validated against a human-rated sample	An LLM judge scores each generated answer against its reference answer and supporting documents using the study's rubric; a sample of its scores is checked against a human rater's scores, with agreement measured via Cohen's	Strict ≥ 80%, lenient ≥ 90%

		K	
Instruction Following	Format-compliance rate (citation presence, referral phrasing, response structure)	Automated constraint-checker script parses each response for required format compliance	Instruction-level $\geq 95\%$, strict prompt-level $\geq 85\%$
Handling of Unsupported Queries	Decline rate and fabricated-citation rate on no-evidence queries	Run the ≈ 25 in-scope, no-evidence queries with the router bypassed and record whether the model declines without attaching a citation	$\geq 90\%$ decline, and no fabricated citation

Table 19

Evaluation Matrix for Objective 4: Retrieval-Augmented Generation (RAG) Performance, Groundedness, and Citation Percision

Evaluation Criterion	Metric	Proposed Testing Method	Target Benchmark
Retrieval Accuracy	Top-5 passage retrieval success rate (Recall@5, MMR)	Check whether the top-5 retrieved passages contain the pooled gold-labeled ground truth	Recall@5 ≥ 0.85; MMR ≥ 0.70
Answer Groundedness	Fact alignment / hallucination rate, answer-level and claim-level	Verify each claim in generated answers directly against retrieved source passages	Answer-level $\geq 85\%$; claim-level $\geq 95\%$

Citation Precision	CItation metadata correctness	Validate cited page numbers, manual titles, and issuing offices	$\geq$ 95% accuracy on cited document sources

Table 20

Evaluation Matrix for Objective 5: Query-Routing Mechanism and Classification Performance

Evaluation Criterion	Metric	Proposed Testing Method	Target Benchmark
Routing Accuracy	Path selection accuracy	Test query classification across Path A (Cited Answer) Path B (Office Referral), and Path C (Out-of-Scope)	Overall $\geq$ 85%; macro-F1 $\geq$ 0.80; dangerous error (out-of-scope answered as in-scope) $\leq$ 5%

Table 21

Evaluation Matrix for Objective 6: Overall System Software Quality Based on ISO/IEC 25010:2023 Standards

Evaluation Criterion	Metric	Proposed Testing Method	Target Benchmark
ISO/IEC 25010:2023 Assessment	Mean Likert score across Functional Suitability, Performance Efficiency, and Reliability characteristics	Administer standardized ISO/IEC 25010 evaluation questionnaire to three to five technical and domain experts	Overall mean rating ≥ 4.0 out of 5.0

Analysis of the Evaluation Criteria

Establishing these evaluation criteria prior to implementation provides a clear roadmap for system construction, optimization, and software quality assurance across all six specific objectives.

1. **Development Target Setting (Objectives 1 & 2):** Metrics like Peak VRAM Usage, Response Delay, and Generation Rate act as operational limits during local development. Imposing a strict limit of 8 GB VRAM ensures that 4-bit NormalFloat (NF4) quantization and QLoRA adapter designs continue to be efficient for resource-limited, consumer-level devices. Mandating an empirical 10-query calibration run on the final deployment device before testing eliminates post-hoc modifications and guarantees that latency assertions stay reproducible. Furthermore, creating a well-organized knowledge base with clear document titles, issuing agencies, publication years, and page numbers sets the foundational metadata necessary for precise downstream citation.
2. **Quality Assurance in Fine-Tuning and RAG Design (Objectives 3 & 4):** Criteria covering Answer Correctness, Instruction Following, Retrieval Accuracy, Answer Groundedness, and Citation Precision dictate how behavioral fine-tuning, text chunking, embedding generation, and vector retrieval pipelines must be engineered. Evaluating model behavior through LLM scoring paired with human validation (measured via Cohen's κ) ensures rigorous answer quality. Furthermore, setting high targets for groundedness and citation metadata accuracy forces the system to rely strictly on retrieved source passages and output verifiable document references.
3. **Scope Control and Reliability (Objective 5):** Routing Accuracy and Handling of Unsupported Queries establish safety and operational boundaries. Evaluating query classification across Path A (Cited Answer), Path B (Office Referral), and Path C (Out-of-Scope) via a 3×3 confusion matrix prevents the system from attempting to answer out-of-scope or unevidenced prompts. Enforcing a zero-tolerance policy for fabricated citations on no-evidence queries guarantees safe rejection behavior when supporting context is absent.
4. **Standardized Quality Evaluation (Objective 6):** Finally, evaluating the prototype using selected characteristics from ISO/IEC 25010:2023 provides a

comprehensive assessment of the system's overall software quality. Through expert and end-user assessment of functional suitability, performance efficiency, and reliability, the research determines if the integrated system aligns with acceptable software standards for academic use. Utilizing statistical metrics like Wilson 95% Confidence Intervals within small query categories (15 to 25 items) guarantees that single-item variability does not undermine statistical inferences during the final assessment.

Table 22

Routing Confusion Matrix (Expected Route × Actual Route)

	Actual: Cited Answer	**Actual: Office Referral**	**Actual: Out-of-Scope**
Expected: Cited Answer (≈40)	✓	miss	miss
Expected: Office Referral (≈25)	miss	✓	miss
Expected: Out-of-Scope (≈15)	⚠ dangerous error	miss	✓

Instrument and Technique Used

This study utilizes a combination of dedicated hardware platforms, open-source software libraries, custom automated evaluation scripts, as well as formal methodology in order to design, implement, as well as evaluate the local RAG system.

Hardware and Execution Environment

The entire system development, model fine-tuning, retrieval indexing, and evaluation processes are implemented on a single dedicated local machine in order to maintain consistent runtime conditions.

1. **Hardware Specification:** The development and evaluation are conducted on a single custom-built desktop by using NVIDIA GeForce RTX 4060, AMD Ryzen 5 5600 (6 cores/ 12 threads) and 16 GB DDR4-3200 RAM. The usage of the strict upper limit of 8 GB VRAM is one of the major structural limitations of the study to ensure lightweight local execution.
2. **Operating Environment:** The system operates within Windows Subsystem for Linux 2 (WSL2), providing a native Linux toolchain for Python execution and CUDA-accelerated GPU management.

Table 23

Software Instruments and Frameworks

Instrument / Tool	Version / Model	Functional Role
Python	3.13 (Virtual Env)	Core programming language for end-to-end pipeline implementation, orchestration, and statistical evaluation.
Llama 3.2 3B	4-bit NF4 Quantized	Primary large language model utilized for local, offline answer generation without external API dependencies.
transformers & bitsandbytes	Latest stable	Enables 4-bit NormalFloat (NF4) quantized model loading and memory-efficient local inference.
peft (QLoRA)	Latest stable	Parameter-Efficient Fine-Tuning framework used to train behavioral low-rank adapters on the generator model.

rank_bm25	Latest stable	Lexical retrieval engine providing sparse keyword matching over document chunks.
sentence-transformers	`bge-small-en-v1.5` (384-d, CPU)	Dense retrieval embedding model for vector search and nearest-neighbor scope check lookup.
NumPy	Latest stable	Vector math operations, Reciprocal Rank Fusion (RRF) score aggregation, and metric computations.
pytest	Latest stable	Automated testing framework executing unit tests and fake-retriever stubs across system components.
Constraint-Checker Script	Group-built (IFEval-style)	Custom automated parser evaluating instruction-following compliance and refusal handling on no-evidence queries.
Runtime Instrumentation	`torch.cuda`, `wall-clock timers`, `nvidia-smi`	Profiling tools monitoring real-time VRAM allocation, time-to-first-token, generation throughput, and total latency.
Git	Standard VCS	Version control system enforcing systematic commit protocols and tagging frozen experimental artifacts.

Research Techniques and Methodological Controls

To guarantee empirical rigor, reproducibility, and unbiased evaluation without a comparative baseline, the study incorporates eight core methodological techniques:

1. **Stratified Sampling:** The 80-query evaluation set is divided across three routing categories (≈40 Cited Answer, ≈25 Office Referral, and ≈15 Out-of-Scope) so that each route has enough items for its proportion to be interpretable, reported with a 95% confidence interval.
2. **Held-Out Splitting:** Fine-tuning dataset items used during QLoRA adapter training are strictly isolated from the evaluation dataset to test model generalization rather than memorization.
3. **Threshold Pre-Registration:** Target performance thresholds across all ten primary metrics are fixed prior to evaluation execution, preventing post-hoc target adjustment.
4. **Pooled Relevance Judgment:** Ground-truth document chunk labels are evaluated at a depth of k=5, directly matching the maximum passage depth supplied to the generator model.
5. **Evaluation Scoring Protocol:** Metrics that can be computed directly (response time, speed, memory, retrieval accuracy, routing accuracy) are calculated automatically from system logs. Metrics needing judgment (answer correctness, groundedness, citation accuracy) are scored first by an LLM judge using the study's rubric. A sample of its scores is checked against a human rater's scores on the same answers, and their agreement is measured using Cohen's Kappa (κ) [37]. Cases the LLM judge is unsure about are decided by a human rater.
6. **Deterministic Execution:** All evaluation runs enforce zero-temperature generation (T=0), fixed random seeds, and frozen corpus indices so that

outputs are expected to repeat across runs; repeatability is checked in Test 6.3 rather than assumed, since GPU floating-point arithmetic can still introduce small run-to-run variation even under fixed settings.

7. **Pre-Validation Routing Harness:** The routing sub-system is validated end-to-end during system assembly prior to final reporting to ensure live signal stability.
8. **Deterministic Backstop Logic:** Citation validity and strict system constraints are enforced through explicit programming logic rather than relying solely on probabilistic language model generation.

References

[1] Castulo, N.J., Marasigan, A.C., Buenaventura, M.L.D. *et al.* Contextualizing the challenges of education graduate students in the Philippines: translating needs analysis into strategic solutions. *Discov Educ* **4**, 27 (2025). https://doi.org/10.1007/s44217-025-00416-7

[2] Swacha, J., & Gracel, M. (2025). Retrieval-Augmented Generation (RAG) Chatbots for Education: A Survey of Applications. *Applied Sciences*, *15*(8), 4234. https://doi.org/10.3390/app15084234

[3] M. A. Hanif Shaikh and A. Al Shafi, "Towards Developing a Multimodal Chat Assistant for University Stakeholders: RAG-Based Approach," *2025 28th International Conference on Computer and Information Technology (ICCIT)*, Cox's Bazar, Bangladesh, 2025, pp. 5329-5334, doi: 10.1109/ICCIT68739.2025.11490128.

[4] M. Saman, G. A. F. Alfarisy, R. Amelia, and N. R. Fadhliana, “A RAG-Based Academic Information Chatbot Using Lightweight LLaMA and Indo-Sencence-BERT”, *IJAIDM*, vol. 8, no. 3, pp. 687–696, Dec. 2025, doi: 10.24014/ijaidm.v8i3.38150.

[5] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. The Eleventh International Conference on Learning Representations. https://doi.org/10.48550/arXiv.2210.17323

[6] Meta. (2024). Llama 3.2 model card. GitHub. llama-models/models/llama3_2/MODEL_CARD.md at main · meta-llama/llama-models

[7] Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2024). AWQ: Activation-aware weight quantization for

on-device LLM compression and acceleration. Proceedings of Machine Learning and Systems, 6, 87–100. https://doi.org/10.48550/arXiv.2306.00978

[8] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. Advances in Neural Information Processing Systems (NeurIPS) 36, 10088–10115. https://doi.org/10.48550/arXiv.2305.14314

[9] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.
[2005.11401] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

[10] Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, 150–158.
https://doi.org/10.48550/arXiv.2309.15217

[11] International Organization for Standardization. (2023). Systems and software engineering—Systems and software quality requirements and evaluation (SQuaRE)—Product quality model (ISO/IEC Standard No. 25010:2023). ISO/IEC 25010:2023 - Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model

[12] Geifman, Y., & El-Yaniv, R. (2017). Selective classification for deep neural networks. Advances in Neural Information Processing Systems, 30, 4878–4887. [ ](https://proceedings.neurips.cc/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html)

[13] Rajpurkar, P., Jia, R., & Liang, P. (2018). Know what you don't know: Unanswerable questions for SQuAD. Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), 784–789. https://doi.org/10.48550/arXiv.1806.03822

[14] J. Chen, Y. Shabanzadeh, E. Crnčević, T. Hoefler, and D. Alistarh, "The geometry of LLM quantization: GPTQ as Babai's nearest plane algorithm," arXiv preprint arXiv:2507.18553, 2025. [Online]. Available: https://arxiv.org/abs/2507.18553

[15] Z. Liu et al., "ParetoQ: Improving scaling laws in extremely low-bit LLM quantization," in Proc. Adv. Neural Inf. Process. Syst. 38 (NeurIPS), 2025. [Online]. Available: https://arxiv.org/abs/2502.02631. [Accessed: Aug. 15, 2026].

[16] L. D. Brown, T. T. Cai, and A. DasGupta, "Interval estimation for a binomial proportion," Statist. Sci., vol. 16, no. 2, pp. 101–133, 2001, doi: 10.1214/ss/1009213286.

[17] C. Zhou et al., "LIMA: Less is more for alignment," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023, pp. 55006–55021. [Online]. Doi: 10.52202/075280-2400

[18] H. Rauf, U. Khan, J. A. Bashatah, and A. Zaib, "Efficient multilingual spam detection on resource-constrained devices: A comparative analysis of QLoRA fine-tuning of Gemma 3, Qwen 3, and Llama 3.2 models," Complex Intell. Syst., vol. 12, art. no. 119, Feb. 2026, doi: 10.1007/s40747-026-02247-x.

[19] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou, "Instruction-following evaluation for large language models," arXiv preprint arXiv:2311.07911, 2023. [Online]. Available: https://arxiv.org/abs/2311.07911

[20] M. Qin, Q. Sun, T. French, and W. Liu, "Enhancing RAG system performance through semantic layout chunking," in AI 2025: Advances in Artificial Intelligence (Lecture Notes in Computer Science), vol. 16370, Cham, Switzerland: Springer, 2025, pp. 28–39, doi: 10.1007/978-981-95-4969-6_3.

[21] T. Pham and L. N. Phuong, "Optimizing context retrieval for RAG via heading-aware chunking and hierarchical document structure integration," in Proc. 2025 IEEE 7th Symp. Comput. & Informat. (ISCI), 2025, doi: 10.1109/ISCI65687.2025.11167620.

[22] C. Kiss, M. Nagy, and P. Szilágyi, "Max–Min semantic chunking of documents for RAG application," Discover Comput., vol. 28, art. no. 117, Jun. 2025, doi: 10.1007/s10791-025-09638-7.

[23] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR), Boston, MA, USA, 2009, pp. 758–759. [Online]. Available: https://doi.org/10.1145/1571941.1572114. [Accessed: Aug. 15, 2026].

[24] Y. Wang, G. Dai, S. Ke, and C. Zheng, "Evaluating sparse and dense retrieval in retrieval-augmented generation systems: A study," in Proc. 2024 10th Int. Conf. Communication and Information Processing (ICCIP), Lingshui, Hainan, China, 2024, pp. 548–554. [Online]. Available: https://doi.org/10.1145/3708657.3708747. [Accessed: Aug. 13, 2026].

[25] I. van Dort and M. Heuss, "How do LLMs cite? A mechanistic interpretation of attribution in RAG," in Advances in Information Retrieval (ECIR 2026) (Lecture Notes in Computer Science), Cham, Switzerland: Springer, 2026, pp. 458–473, doi: 10.1007/978-3-032-21324-2_35.

[26] H. Zhang and W. Wu, "Do LLMs know when evidence is insufficient? An evidence sufficiency benchmark for answer-abstention calibration in retrieval-augmented generation," Computers, Materials & Continua, vol. 89, no. 1, art. no. 69, 2026. [Online]. Available: https://doi.org/10.32604/cmc.2026.086343. [Accessed: Aug. 14, 2026].

[27] G. K.-M. Liu, B. Li, A. Cohan, W. G. Walden, and E. Yang, "Investigating retrieval-augmented generation systems on unanswerable, uncheatable, realistic, multi-hop queries," in Lecture Notes in Computer Science: Advances in Information Retrieval, Springer Nature Switzerland, 2026, pp. 360–370. [Online]. Available: https://doi.org/10.1007/978-3-032-21300-6_26. [Accessed: Aug. 14, 2026].

[28] Peyton, K., & Unnikrishnan, S. (2022). A comparison of chatbot platforms with the state-of-the-art sentence BERT for answering online student FAQs. Results in Engineering. https://doi.org/10.1016/j.rineng.2022.100856

[29] Bilquise, G., Ibrahim, S., & Shaalan, K. (2022). Bilingual AI-Driven Chatbot for Academic Advising. International Journal of Advanced Computer Science and Applications. https://doi.org/10.14569/ijacsa.2022.0130808

[30] Essop, L. (2024). Enhancing the usability of a university student support services FAQ Chatbot. https://doi.org/10.51415/10321/5571

[31] Tamascelli, M., Bunch, O., Fowler, B., Taeb, M., & Cohen, A. (2025). Academic Advising Chatbot Powered with AI Agent. Proceedings of the 2025 ACM Southeast Conference. https://doi.org/10.1145/3696673.3723065

[32] Zhang, W., & Zhang, J. (2025). Hallucination Mitigation for Retrieval-Augmented Large Language Models: A Review. Mathematics. https://doi.org/10.3390/math13050856

[33] Tjandra, B. A., Razzak, M., Kossen, J., Handa, K., & Gal, Y. (2024). Fine-Tuning Large Language Models to Appropriately Abstain with Semantic Entropy. ArXiv, abs/2410.17234. https://doi.org/10.48550/arxiv.2410.17234

[34] Magesh, V., Surani, F., Dahl, M., Suzgun, M., Manning, C.D. and Ho, D.E. (2025), Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools. J Empir Leg Stud, 22: 216-242. https://doi.org/10.1111/jels.12413

[35] Neumann, A., Yin, Y., Sowe, S. K., Decker, S., & Jarke, M. (2025). An LLM-Driven Chatbot in Higher Education for Databases and Information Systems. IEEE Transactions on Education, 68, 103-116. https://doi.org/10.1109/te.2024.3467912

[36] Wen, B., Yao, J., Feng, S., Xu, C., Tsvetkov, Y., Howe, B., & Wang, L. L. (2024). Know Your Limits: A Survey of Abstention in Large Language Models. Trans. Assoc. Comput. Linguistics, 13, 529-556. https://doi.org/10.1162/tacl_a_00754

[37] J. Cohen, "A coefficient of agreement for nominal scales," Educ. Psychol. Meas., vol. 20, no. 1, pp. 37–46, 1960, doi: 10.1177/001316446002000104.

[38] Lagman, A., Ramirez, I. C. R., Esteban, A. P., Rivera, R. P. L., Santos, R. D., & Juliano, J. (2024). Development and Evaluation of an Enterpriselevel Information System for Digital Governance in Philippine SUCs Using Agile Software Methodology and ISO/IEC 25010 Software Quality Model. 2024 IEEE 16th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM), 1-6. https://doi.org/10.1109/hnicem64917.2024.11258693

[39] T. Kwok, C. H. Chen, and A. Ramos, "A Multi-Criteria Software Quality Evaluation of AI Meeting Assistants for English-Medium University Lectures Using ISO/IEC 25010 and TOPSIS ," International Journal of Information and Education Technology., vol. 34, no. 1, pp. 88–105, 2026. https://doi.org/10.18178/ijiet.2026.16.1.2479

[40] Barletta, V. S., Caivano, D., Colizzi, L., Dimauro, G., & Piattini, M. (2022). Clinical-chatbot AHP evaluation based on "quality in use" of ISO/IEC 25010. International journal of medical informatics, 170, 104951. https://doi.org/10.2139/ssrn.4041111

[41] Rojas, H., Renteria, R., Martinez Duran, V., Gutierrez, Y., Cabrera, M., & Aminuddin, A. (2025). Mapping the Evolution and Future Directions of ISO/IEC 25010: A Bibliometric and Thematic Analysis. *Engineering, Technology & Applied Science Research*. 2025. https://doi.org/10.48084/etasr.11772

[42] Ayo, E., Steven, F., Dungca, Z., Garduque, E. R. M., Reyes, G. C. P., & Berber, M. B. (2025). Development of Alakdan: An A.I. virtual assistant for Centro Escolar University processes. World Journal of Advanced Research and Reviews. https://doi.org/10.30574/wjarr.2025.25.2.0389.

[43] M. L. Castillo et al., "Assesing AI-Driven Psychological First Aid: A Case Study of SymptoBot Using ISO/IEC 25010 and Technology Acceptance Model (TAM)," 2025 International Workshop on Artificial Intelligence and Education (WAIE),

Yokohama, Japan, Japan, 2025, pp. 1-5,https://doi.org/10.1109/waie67422.2025.11381219 .

[44] Pastorfide, D. M., & Vásquez, J. (2025). AI CHATBOTS INTEGRATION IN ONLINE SHOP IN NUEVA ECIJA, PHILIPPINES: STUDY UTILIZING THE EXPECTATION CONFIRMATION THEORY AND ISO/IEC 25010 CRITERIA. *International Journal of Engineering Applied Sciences and Technology*. https://doi.org/10.33564/ijeast.2025.v09i12.003

[45] Amazon Technologies, Inc., “US 12,353,469 B1,” U.S. Patent, filed Jun. 28, 2024, granted Jul. 8, 2025. [Online]. Available: https://patents.google.com/patent/US12353469B1/en

[46] AdmitHub PBC, “US 11,128,579 B2,” U.S. Patent, filed Sep. 29, 2017, granted Sep. 21, 2021. [Online]. Available: https://patents.google.com/patent/US11128579B2/en

[47] Touvron, H., Martin, L., Stone, K., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. ArXiv. https://doi.org/10.48550/arXiv.2307.09288

[48] Abdin, M., Aneja, J., Awadallah, A., et al. (2024). Phi-3 technical report: A highly capable language model locally on your phone. ArXiv. https://doi.org/10.48550/arXiv.2404.14219

[49] J. Walusimbi, A. M. Oguti, J. B. Ssentongo, and K. Ainebyona, "Arapai: An Offline-First LLM Architecture for Adaptive Learning in Low-Connectivity Environments," arXiv, abs/2603.03339, 2026. [Online]. Available: https://arxiv.org/abs/2603.03339

[50] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," J. Manage. Inf. Syst., vol. 24, no. 3, pp. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.